

LLM Agents



Instructor: Abhishek Aggarwal

Introduction – Abhishek Aggarwal

Senior Engineer, Walmart.



Education

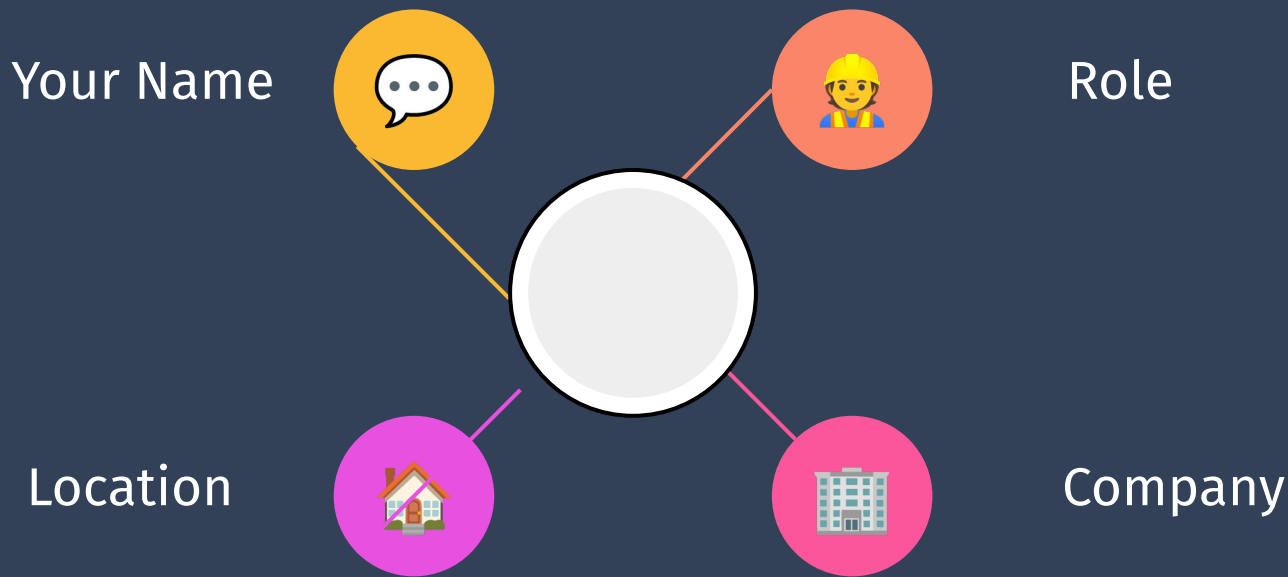


Distributed System + AI



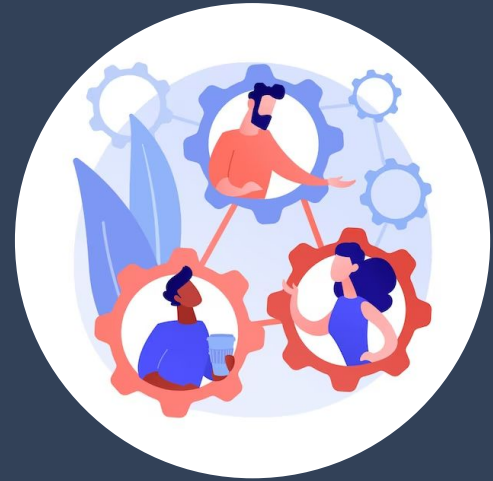
Teaching AI

Welcome to today's class, before we begin, pop into the chat:



Optimize Your Experience

- ✓ Interact with your instructors via Sunday live class.
- ✓ Don't be shy to speak up and get clarifications !



- ✓ Solve your assignments, MCQs and other assessments and get feedback (Thursday review session)
- ✓ Use your resources! It's your experience – what you put in is what you'll get out.

Learning Objectives

- Evolution of AI systems
- What are LLMs? Limitations of LLMs
- Introduction to RAG
- What are Agents?
- LLM Optimization
- Deploy your LLM-based application



Let's Get Started

Conventionally How Systems Were Built



Coventional Bots



Conventionally How Systems Were Built

Rule-Based Approach:

- **Simple Logic:** Responding to specific inputs with predefined outputs.
- **Easy to Implement:** Best for scenarios with predictable user inputs.
- **Limitation:** Cannot handle ambiguity, cannot learn or adapt.

"If someone says 'hello', respond with 'Hi there!'"

Conventionally How Systems Were Built

Rule-Based Approach:

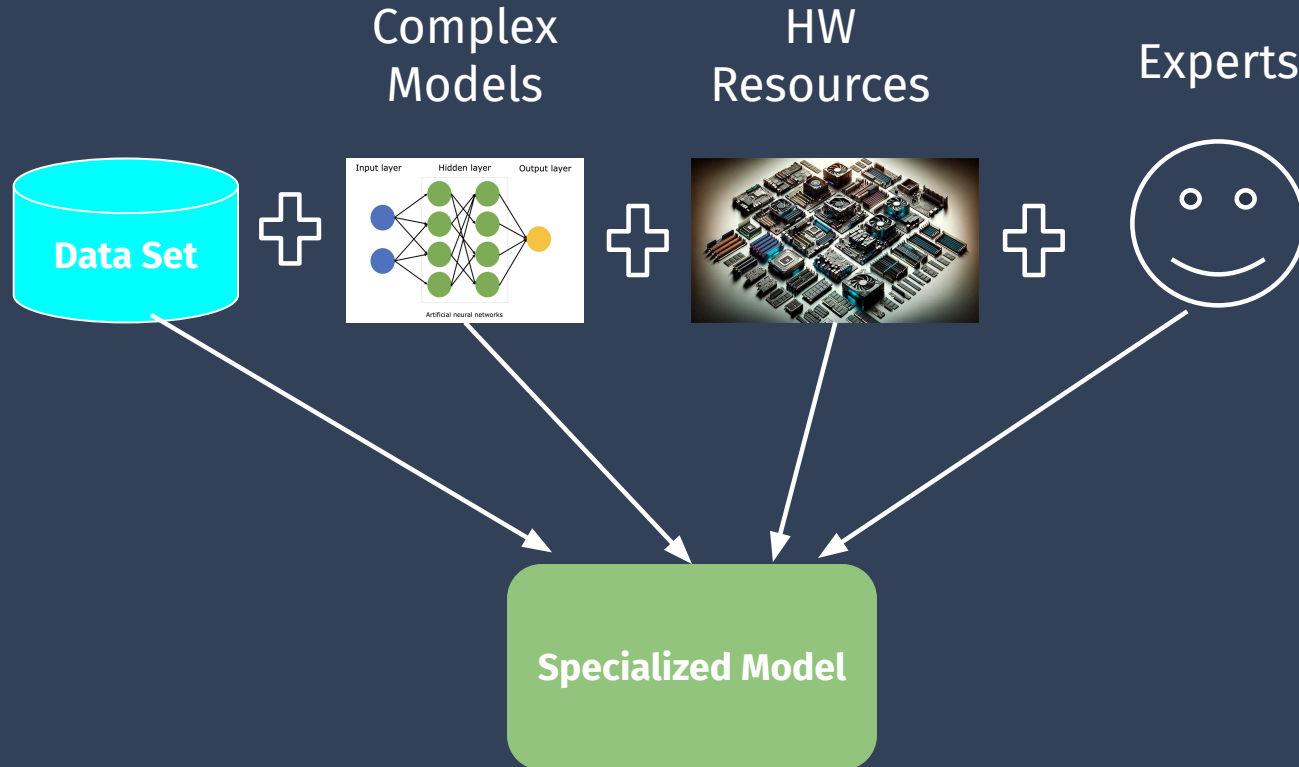
- **Simple Logic:** Responding to specific inputs with predefined outputs.
- **Easy to Implement:** Best for scenarios with predictable user inputs.
- **Limitation:** Cannot handle ambiguity, cannot learn or adapt.

"If someone says 'hello', respond with 'Hi there!'"

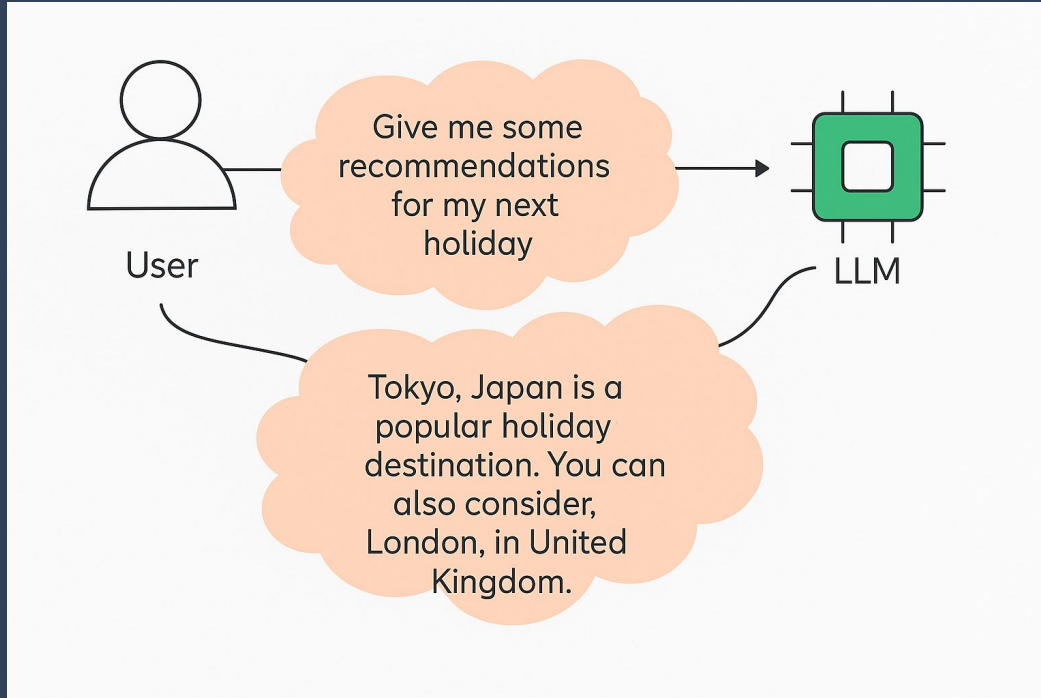
Limitations

- **Non-Scalable:** Requires writing individual rules for each possibility.
- **No Learning:** Cannot adapt over time.
- **Struggles with Ambiguity:** Does not handle complex or nuanced inputs.

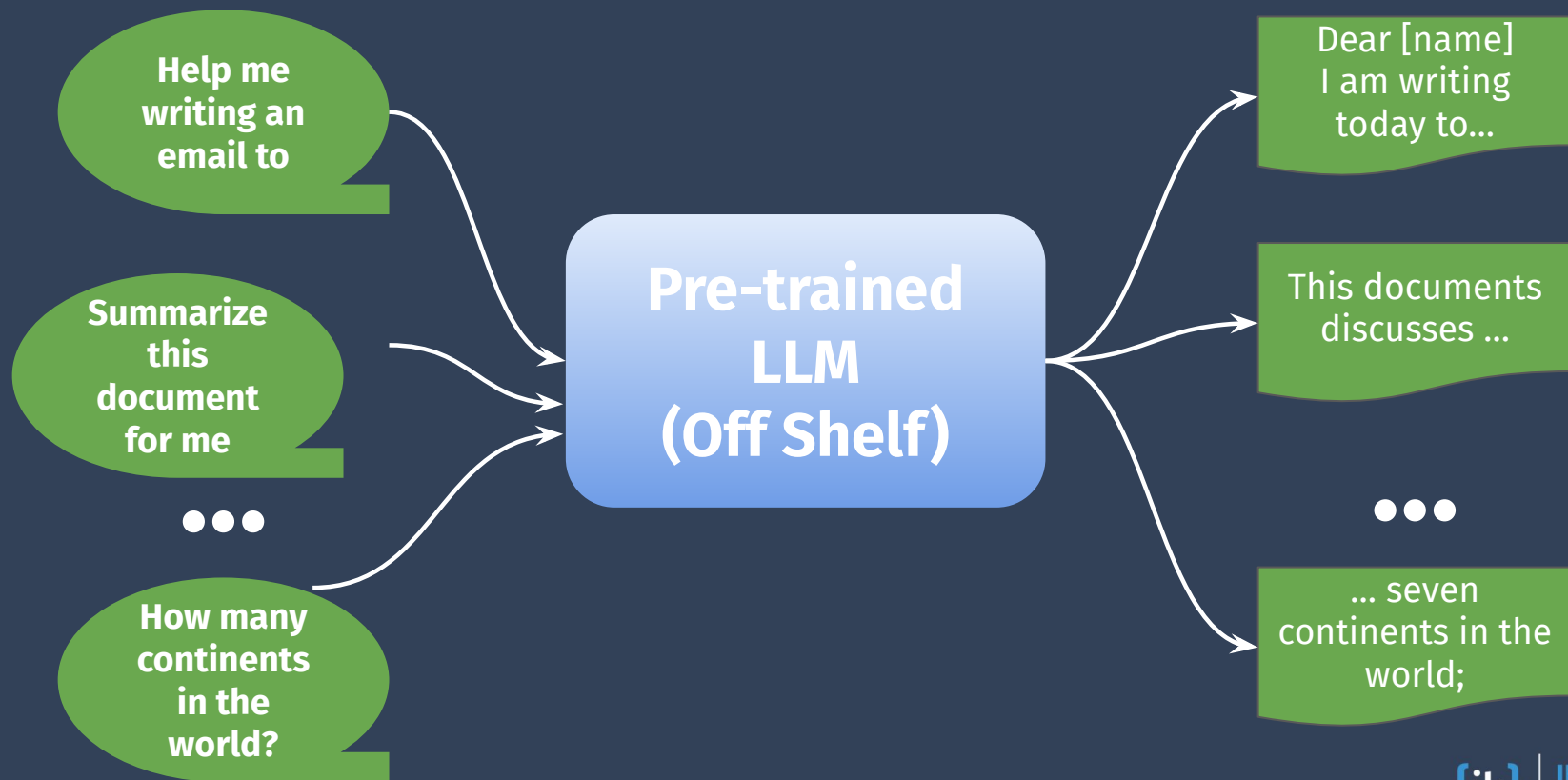
Then Came **ML Model**



Traditional Chatbots to LLM-Powered Chatbots



Large Language Models (LLM)



Large Language Models (LLM)

Solved Many Problems

- Pre-trained on huge amount of data
- Can be used as off shelf model
- Many Cloud APIs are available

Still needs hardware resources ...

LLM

Limitations

- How to use knowledge source
 - Candidates resumes
 - Rules for promotions of Internet planes
 - ...
- **Knowledge cutoff**
 - Who is the CEO of Twitter?

Demo Time



Limitations of LLMs and the Need for RAG

LLM

Example of Limitations

Question: What was uber's revenue in Q1-2025?

If the model was trained in 2024 it will not be able to answer the question

What is in the Prompt?

Answer the following question based only on the provided context:

<context>

Financial and Operational Highlights for First Quarter 2025

- Trips during the quarter grew 18% year-over-year ("YoY")
- Gross Bookings grew 14% YoY to \$42.8 billion, or 18% on a constant currency basis.
- Revenue grew 14% YoY to \$11.5 billion, or 17% on a constant currency basis.

</context>

Question:

What was uber's revenue in Q1-2025?

What is in the Prompt?

Answer the following question based only on the provided context:

<context>

Financial and Operational Highlights for First Quarter 2025

- Trips during the quarter grew 18% year-over-year ("YoY")
- Gross Bookings grew 14% YoY to \$42.8 billion, or 18% on a constant currency basis.
- Revenue grew 14% YoY to \$11.5 billion, or 17% on a constant currency basis.

</context>

Question:

What was uber's revenue in Q1-2025?

**How to get
this context?**



How To Get This Context?

- Retrieval Augmented Generation (RAG)
- Using agents and Tools

How To Get This Context?

- Retrieval Augmented Generation (RAG)
- Using agents and Tools

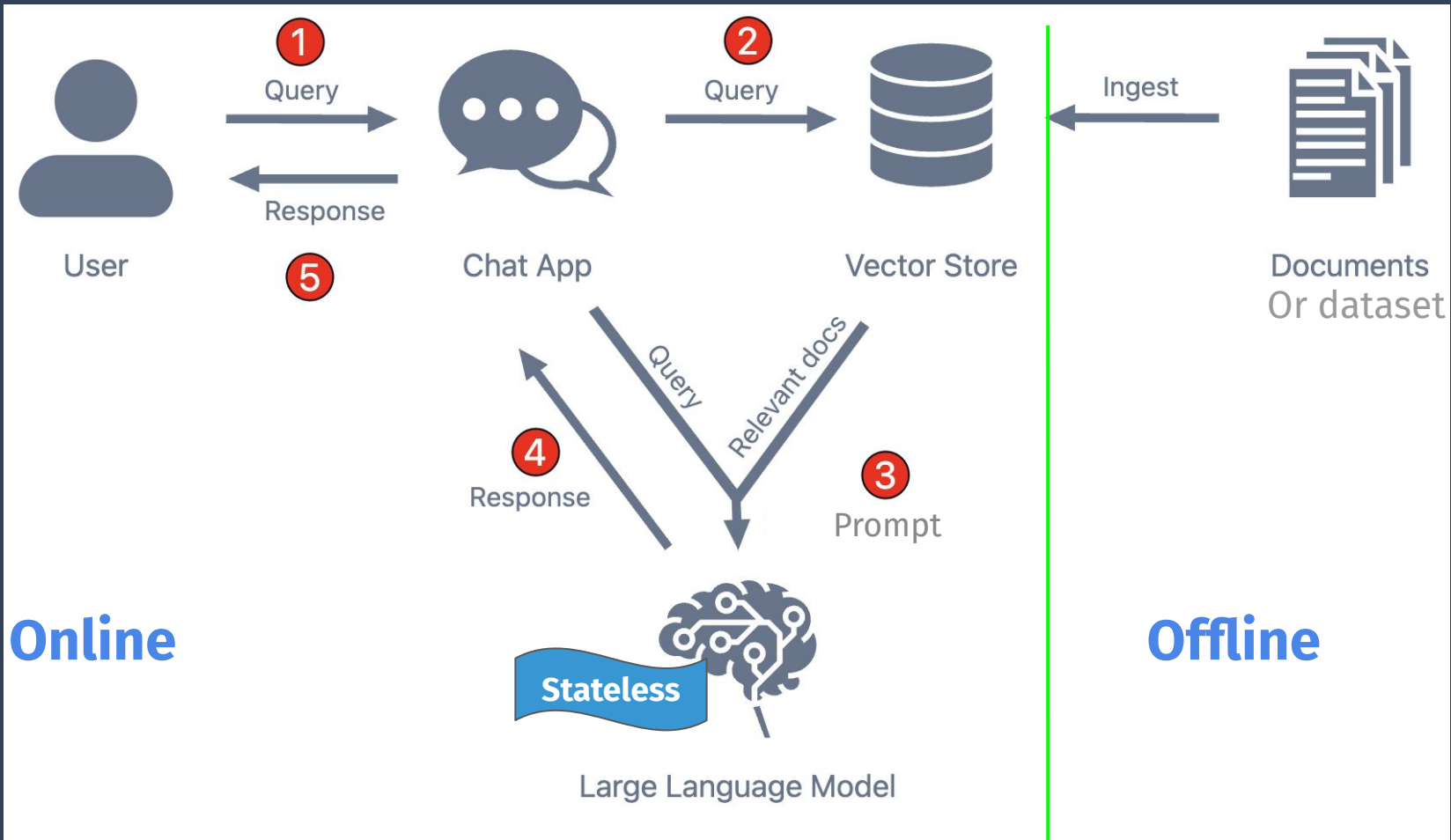
Designing GenAI System

- How to build a GenAI systems?
- Is an LLM alone enough?

Designing GenAI System

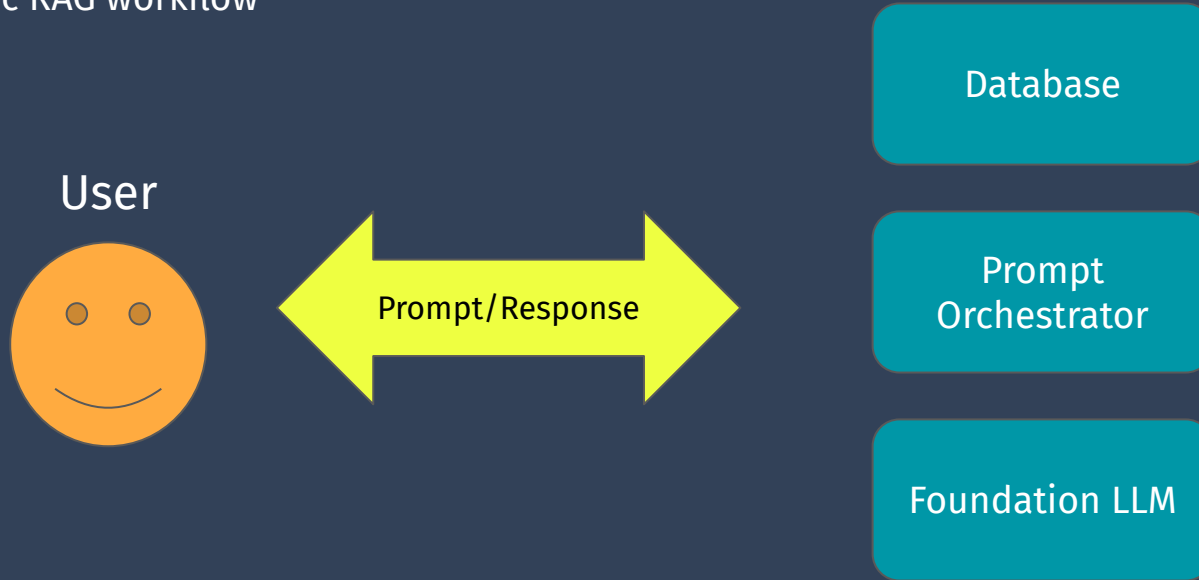
- How to build a GenAI systems?
- Is an LLM alone enough?
 - LLM does not have access to
 - Recent information
 - Domain specific knowledge

Retrieval Augmented Generation RAG



RAG System Architecture

Basic RAG workflow



How RAG Works

Query Processing

The user submits a query.

Retrieval

The retrieval component searches the knowledge base for relevant documents.

Contextual Understanding

The retrieved documents are combined with the query to create a context.

Generation

The language model generates a response based on the context.

What is in the Prompt?

Answer the following question based only on the provided context:

<context>

Financial and Operational Highlights for First Quarter 2025

- Trips during the quarter grew 18% year-over-year ("YoY")
- Gross Bookings grew 14% YoY to \$42.8 billion, or 18% on a constant currency basis.
- Revenue grew 14% YoY to \$11.5 billion, or 17% on a constant currency basis.

</context>

Question:

What was uber's revenue in Q1-2025?

What is in the Prompt?

Answer the following question based only on the provided context:

<context>

Financial and Operational Highlights for First Quarter 2025

- Trips during the quarter grew 18% year-over-year ("YoY")
- Gross Bookings grew 14% YoY to \$42.8 billion, or 18% on a constant currency basis.
- Revenue grew 14% YoY to \$11.5 billion, or 17% on a constant currency basis.

</context>

Question:

What was uber's revenue in Q1-2025?

**How to get
this context?**



Limitations

While RAG systems offer significant advantages, they also have certain limitations:

- Dependency on Knowledge Base Quality
- Retrieval Challenges
- Hallucination
- Bias
- Computational Cost
- Contextual Understanding
- Privacy Concerns

How to overcome these limitations?

Improved Knowledge Base

- Quality Control
- Semantic Representation
- Knowledge Graph Integration

Enhanced Retrieval Algorithm

- Hybrid Retrieval
- Neural Retrieval
- Contextual Retrieval

Advanced Language Models

- Pre-training on Larger Datasets
- Fine-tuning for Specific Tasks
- Multimodal Models

Bias Mitigation

- Diverse Datasets
- Bias Detection & Correction
- Ethical Guidelines

Computational Efficiency

- HW Optimization
- Algorithm Optimization
- Distributed Computing

How To Get This Context?

- Retrieval Augmented Generation (RAG)
- Using agents and Tools

Agent Review

What is in the Prompt?

Answer the following question based only on the provided context:

<context>

Financial and Operational Highlights for First Quarter 2025

- Trips during the quarter grew 18% year-over-year ("YoY")
- Gross Bookings grew 14% YoY to \$42.8 billion, or 18% on a constant currency basis.
- Revenue grew 14% YoY to \$11.5 billion, or 17% on a constant currency basis.

</context>

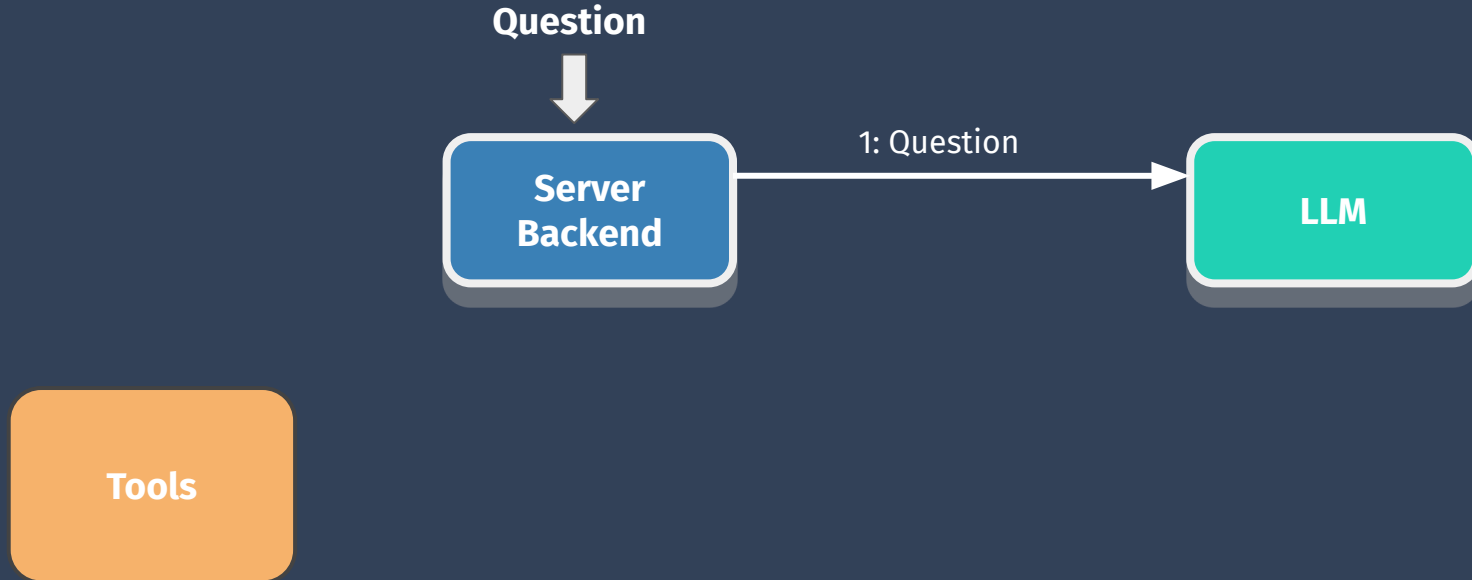
Question:

What was uber's revenue in Q1-2025?

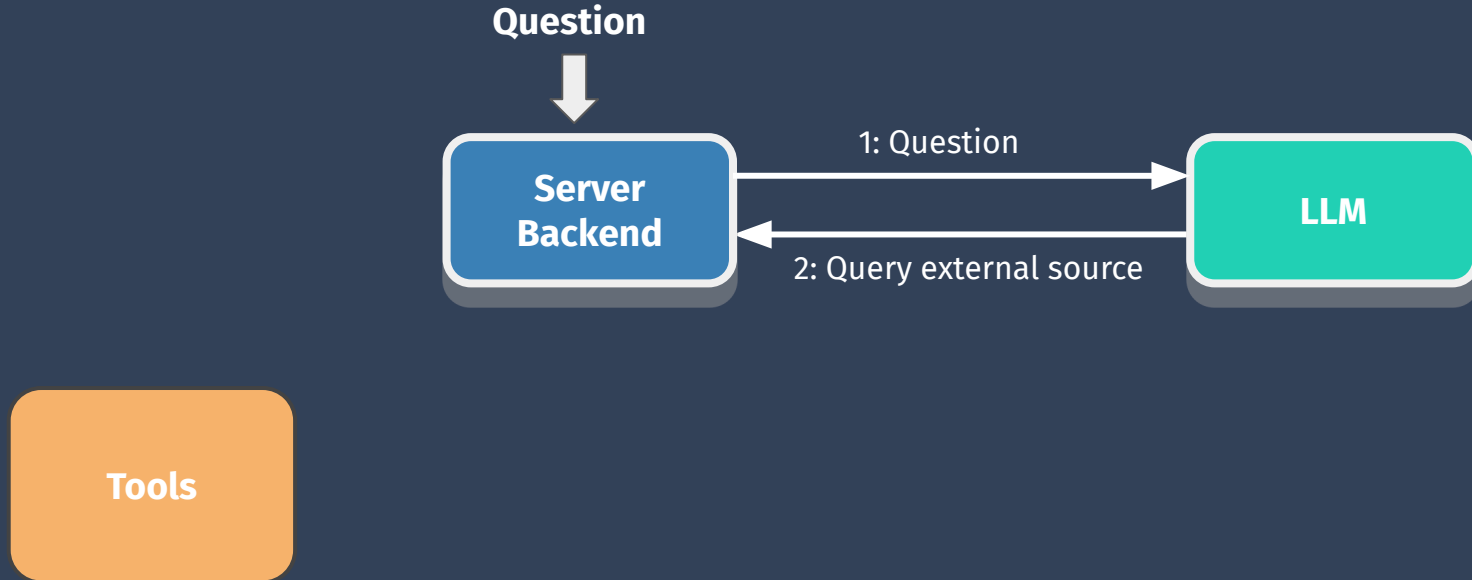
**How to get
this context?**



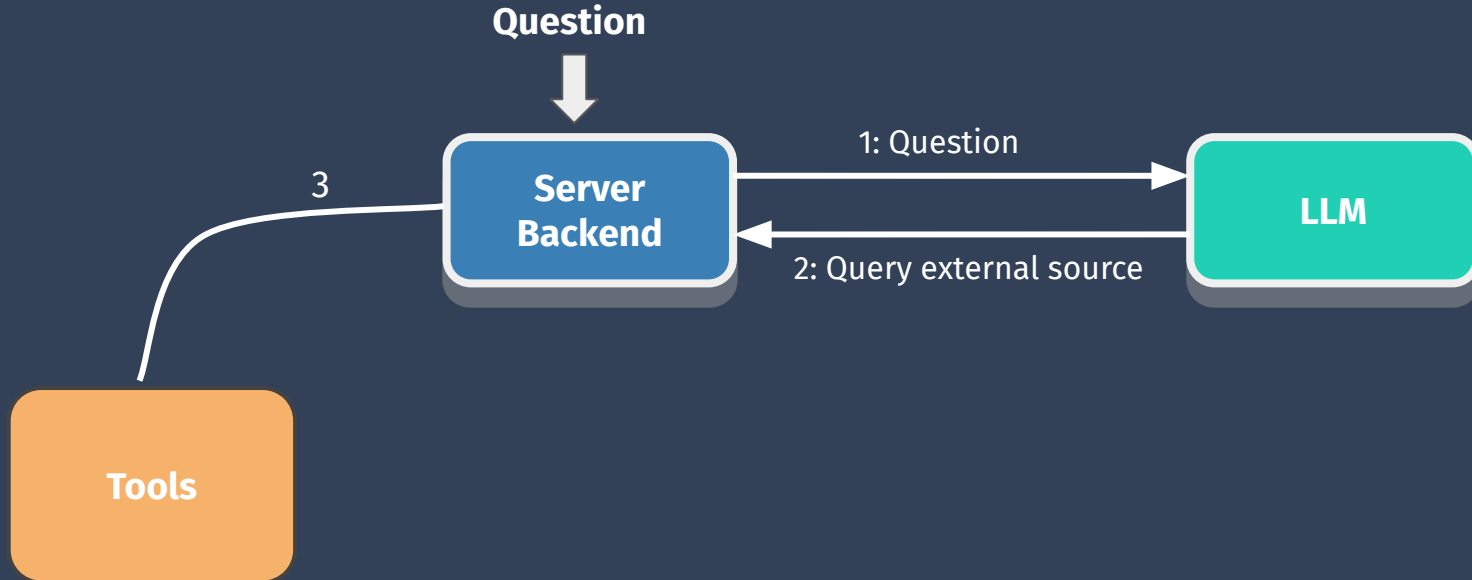
Agent Review



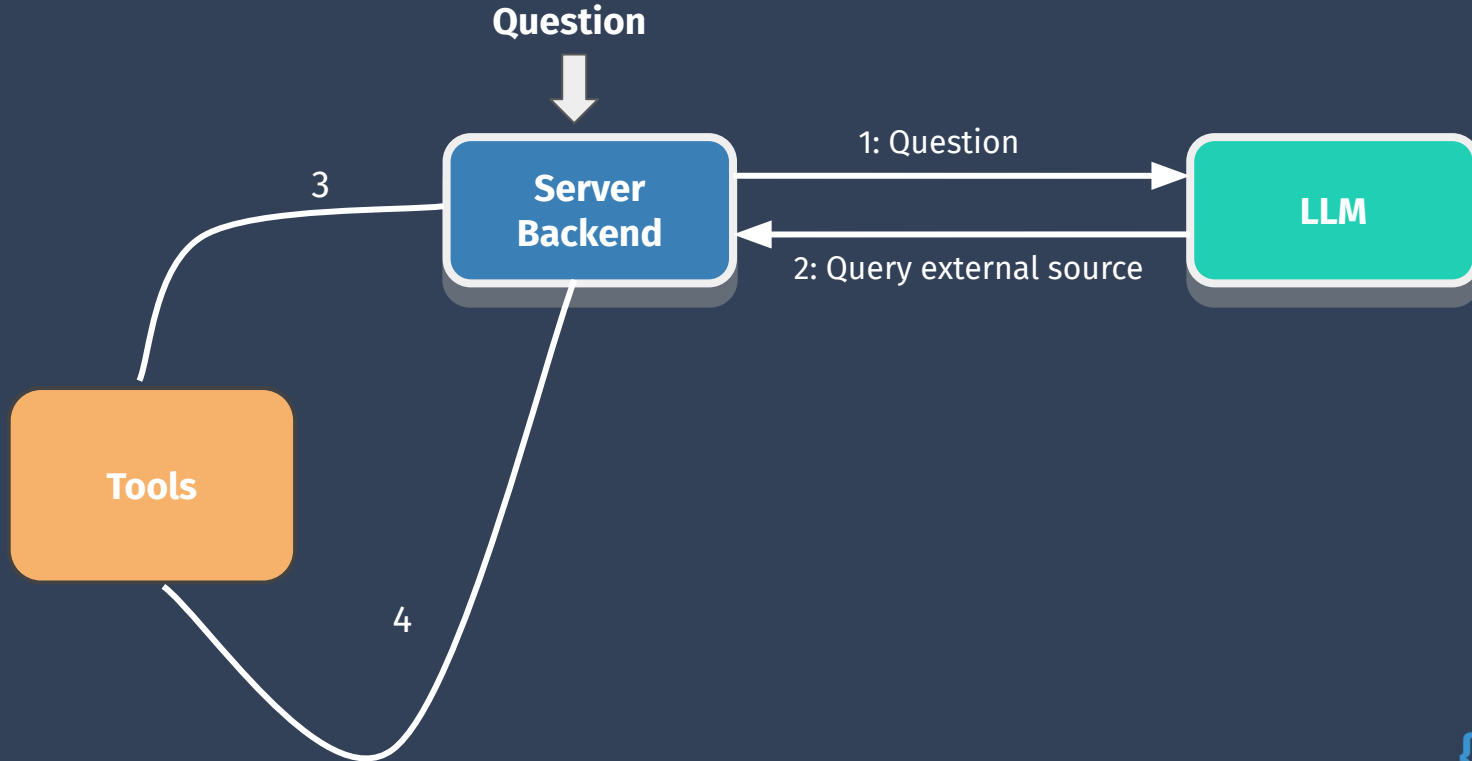
Agent Review



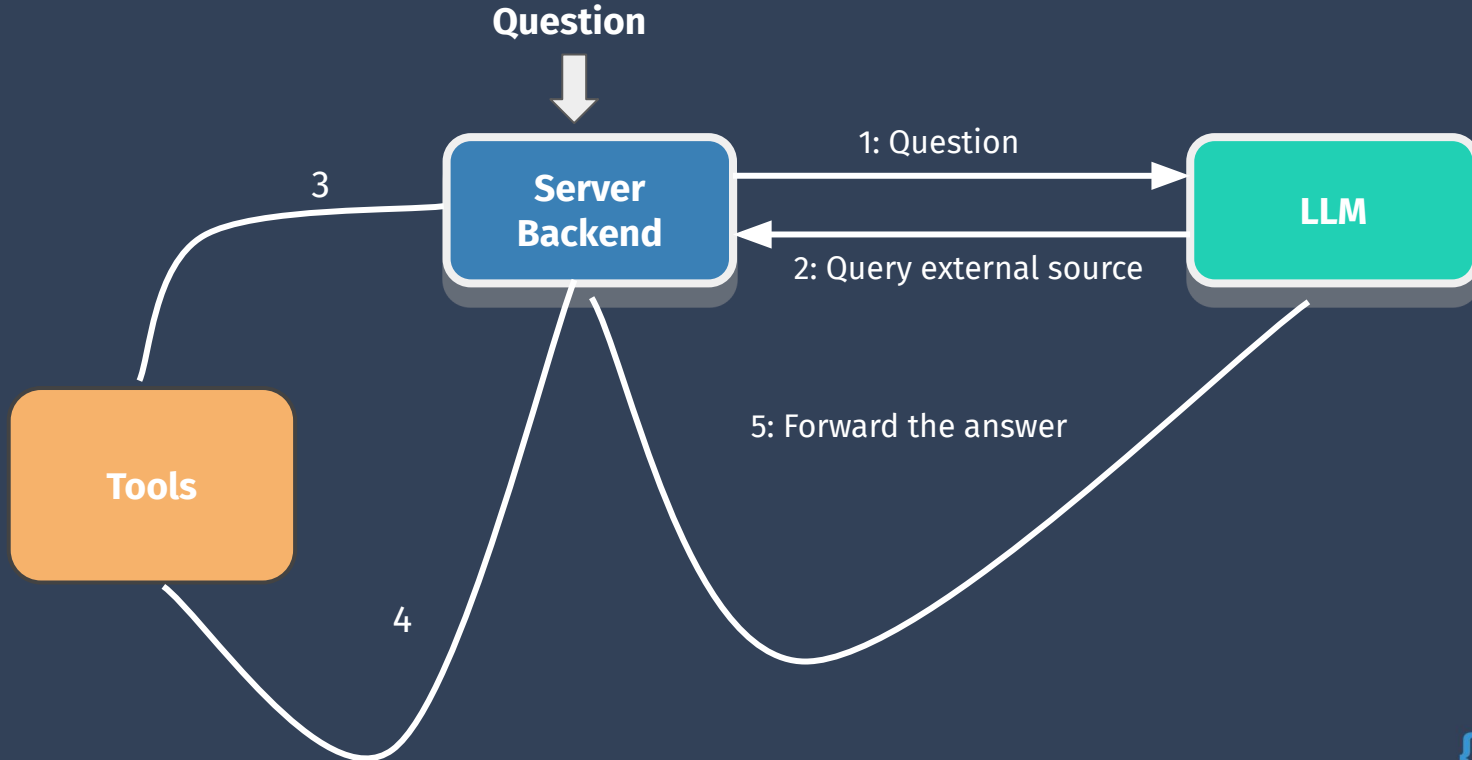
Agent Review



Agent Review

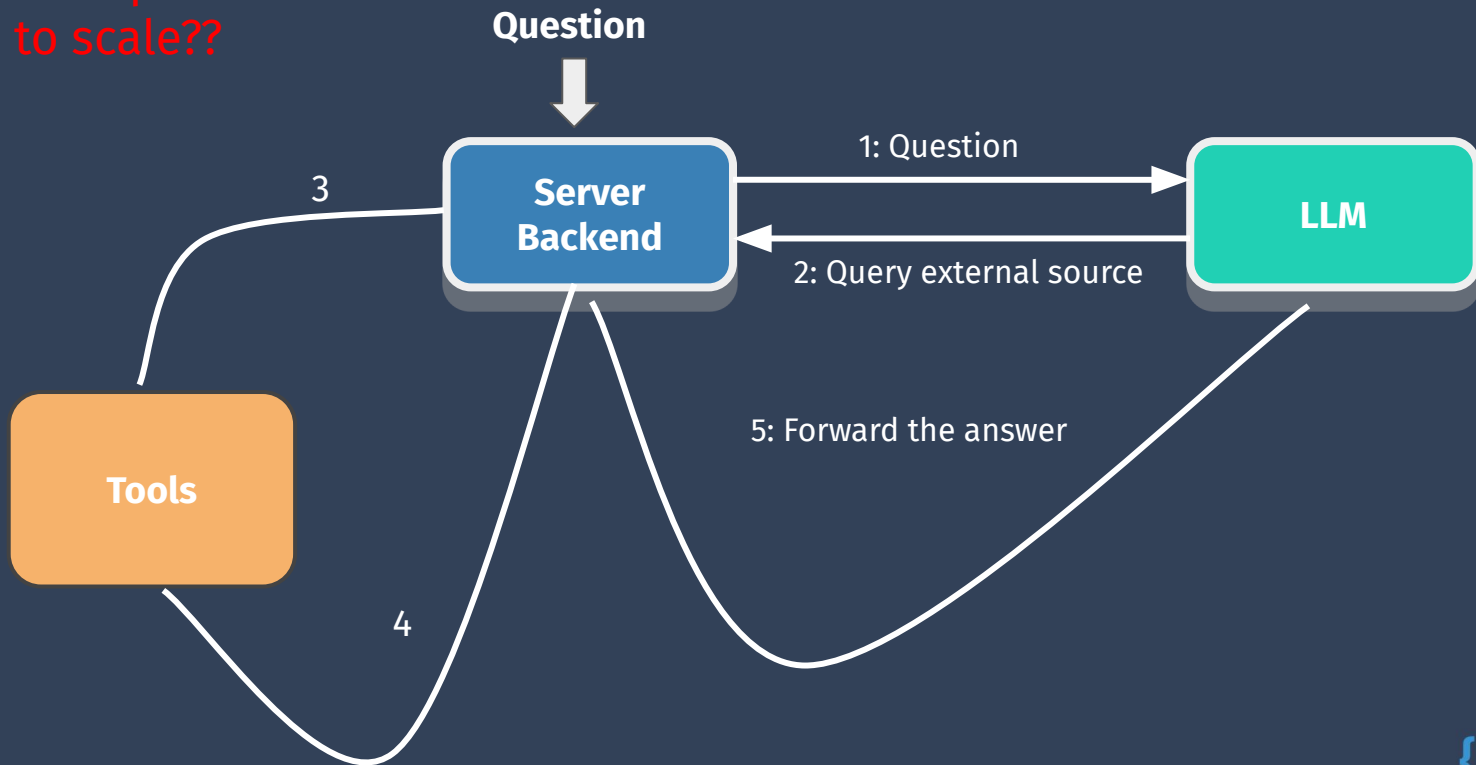


Agent Review



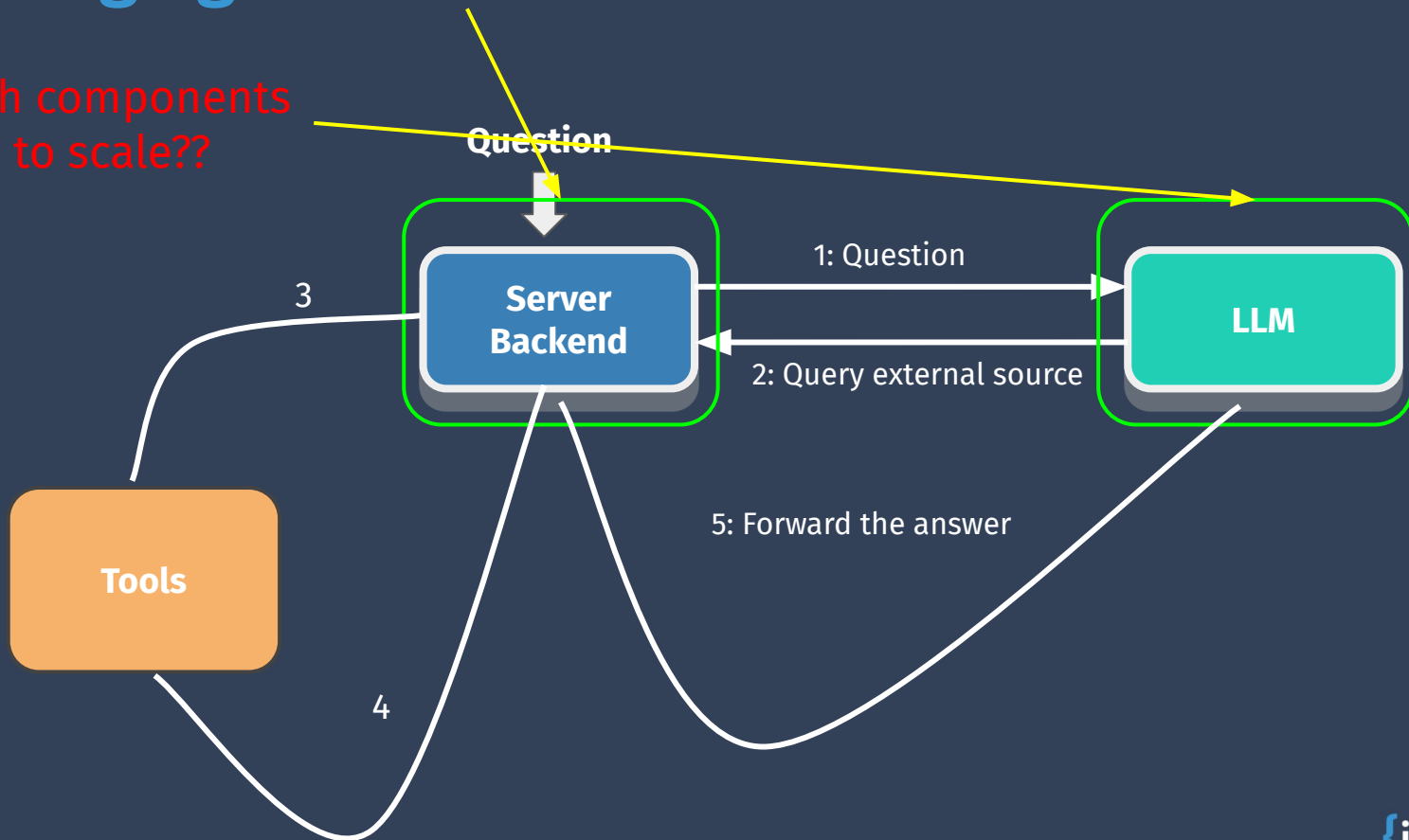
Scaling Agents

Which components
need to scale??



Scaling Agents

Which components need to scale??



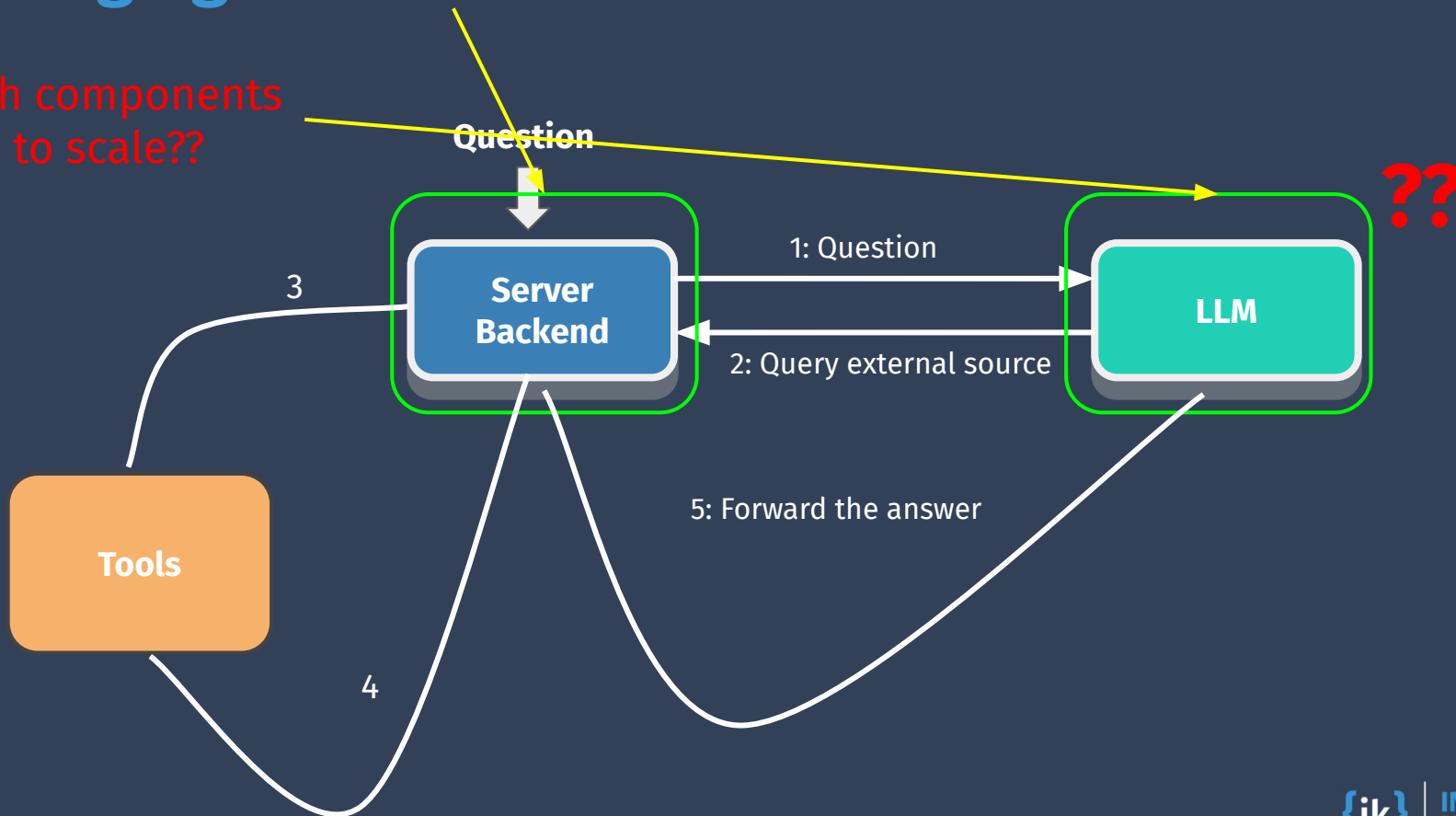
Scaling Agents

Scaling Agents

- Backend and LLMs are scaled independent of each other
- This allows for scaling one without necessarily scaling the other

Scaling Agents

Which components need to scale??



A large, stylized graphic in the center of the slide. It features a yellow, cloud-like shape with a halftone dot pattern. Inside this shape, the words "QUIZ TIME!" are written in a bold, red, outlined font. The word "QUIZ" is on the top line and "TIME!" is on the bottom line. A small, silver megaphone icon is positioned to the left of the word "TIME!".

**QUIZ
TIME!**

Quiz

What are the options for using/deploying LLM

Quiz: Using/Deploying LLM

What are the options for using/deploying LLM

- On the Cloud e.g., openAI
- Locally e.g., using vLLM or Ray server

Local Deployment

Model	Notes
Ollama	Easiest way to run models locally
Transformers + PyTorch	Full control with HuggingFace models
vLLM	High-throughput GPU inference

Local Deployment

Model	Notes
Ollama	Easiest way to run models locally
Transformers + PyTorch	Full control with HuggingFace models
vLLM	High-throughput GPU inference

HuggingFace models → <https://huggingface.co/models>

Ollama models → <https://ollama.com/library>

Let's Run Ollama Locally

- <https://ollama.com/download>
- Start Ollams with from cmd “ollama run gemma3”
- Try it from
 - Command line
 - Through API

Let's Run Ollama Locally

- <https://ollama.com/download>
- Start Ollama with from cmd “ollama run gemma3”
- Try it from
 - Command line
 - Through API

```
response: ChatResponse = chat(model='gemma3', messages=[
    {
        'role': 'user',
        'content': 'How many hours a day? Keep your answer short',
    },
])
print(response['message']['content'])
```

A large, stylized graphic in the center of the slide. It features a yellow, cloud-like shape with a halftone dot pattern. Inside this shape, the words "QUIZ TIME!" are written in a bold, red, bubbly font with white outlines. A small, silver megaphone icon is positioned to the left of the word "TIME!".

**QUIZ
TIME!**

Quiz

What are the pros and cons of local deployment

Cloud Deployment

Option 1: Use Hosted API (OpenAI, Anthropic, Google, etc.)



Pros:

- Zero infrastructure, fully managed, production ready
- Access to recent high quality models



Cons:

- Ongoing cost, no access to weights, limited customization

Cloud Deployment

Option 2: Self-Host Open-Source Models on Cloud Infrastructure

- Use open models (LLaMA, Mistral, etc.) deployed to AWS, GCP, Azure, etc.

A large, stylized graphic in the center of the slide. It features a yellow, cloud-like shape with a halftone dot pattern. Inside this shape, the words "QUIZ TIME!" are written in a bold, red, bubbly font with white outlines. A small megaphone icon is positioned to the left of the word "TIME!".

**QUIZ
TIME!**

Quiz

What are the factors that you will consider to choose from local vs cloud deployment?

Quiz

What are the factors that you will consider to choose from local vs cloud deployment?

- Latency
- Cost
- Maintenance
- Scalability
- Security & Privacy

Scaling LLM

A large, stylized graphic in the center of the page. It features a yellow, cloud-like shape with a halftone dot pattern. Inside this shape, the words "QUIZ TIME!" are written in a bold, red, bubbly font with white outlines. A small, silver megaphone icon is positioned to the left of the word "TIME!".

**QUIZ
TIME!**

Quiz

Which one do you think will be faster local or cloud?

Quiz: Latency

Which one do you think will be faster local or cloud?

	Local	Cloud
Latency	50–200ms	300ms–2s

Quiz: Latency

Which one do you think will be faster local or cloud?

	Local	Cloud
Latency	50–200ms	300ms–2s

- Network Overhead
- Queuing and Throttling
- Cold Starts and Model Re-loading
- Batching and Scheduling Efficiency

A large, yellow, cloud-like shape with a halftone dot pattern is centered on a solid blue background. Inside the yellow shape, the words "QUIZ TIME!" are written in a bold, stylized font. "QUIZ" is in white with a thick red outline, and "TIME!" is in yellow with a thick red outline. A small, silver megaphone icon is positioned to the left of the word "TIME!".

**QUIZ
TIME!**

Quiz

Does Cloud or local cost more?

Quiz: Cost

Does Cloud or local cost more?

- Cloud costs more
- Actual \$\$ depends on the model itself
- GPT-4.1 mini
 - Input: \$0.40 / 1M tokens
 - Output: \$1.60 / 1M tokens
- \$/M tokens local-rented → ~1\$

A large, stylized graphic in the center of the slide. It features a yellow, cloud-like shape with a halftone dot pattern. Inside this shape, the words "QUIZ TIME!" are written in a bold, red, bubbly font with white outlines. A small megaphone icon is positioned between the words "QUIZ" and "TIME!".

**QUIZ
TIME!**

Quiz

Is Maintenance easier on Cloud or local?

Quiz: Maintenance

Is Maintenance easier on Cloud or local?

- No infra maintenance is needed on the cloud while you need to do that on local deployment
- Examples:
 - Autoscaling
 - Fault tolerance

A large, stylized graphic in the center of the slide. It features a yellow, cloud-like shape with a halftone dot pattern. Inside this shape, the words "QUIZ TIME!" are written in a bold, red, bubbly font with white outlines. A small, silver megaphone icon is positioned to the left of the word "TIME!".

**QUIZ
TIME!**

Quiz

Scalability: Cloud or local?

Quiz

Scalability: Cloud or local?

- Cloud solution scales better but tend to be more expensive
- Usually comes with autoscaling

Privacy and Security

- Some domains like financial or health domain have strict requirements around user data
- In these cases local deployment provide much more guarantees as the data never leaves the organization

Cloud Or Local

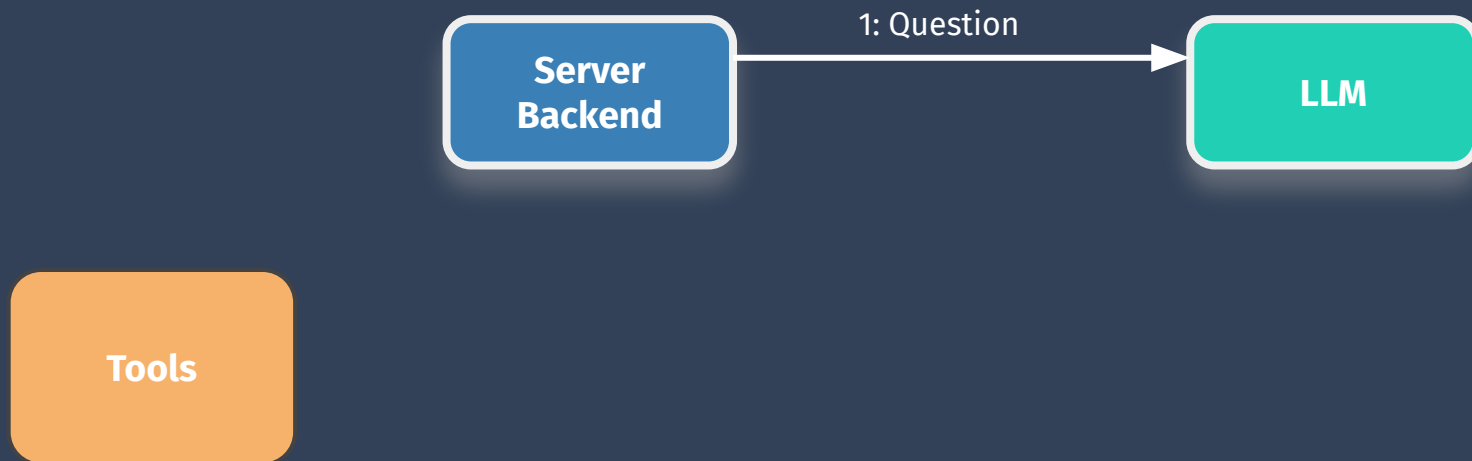
Should You buy it or Build it?

- It depends on your situation
- Local deployment is usually cheaper, faster and more secure
- On the other hand cloud deployment scales more and provides easier setup with little to not maintenance

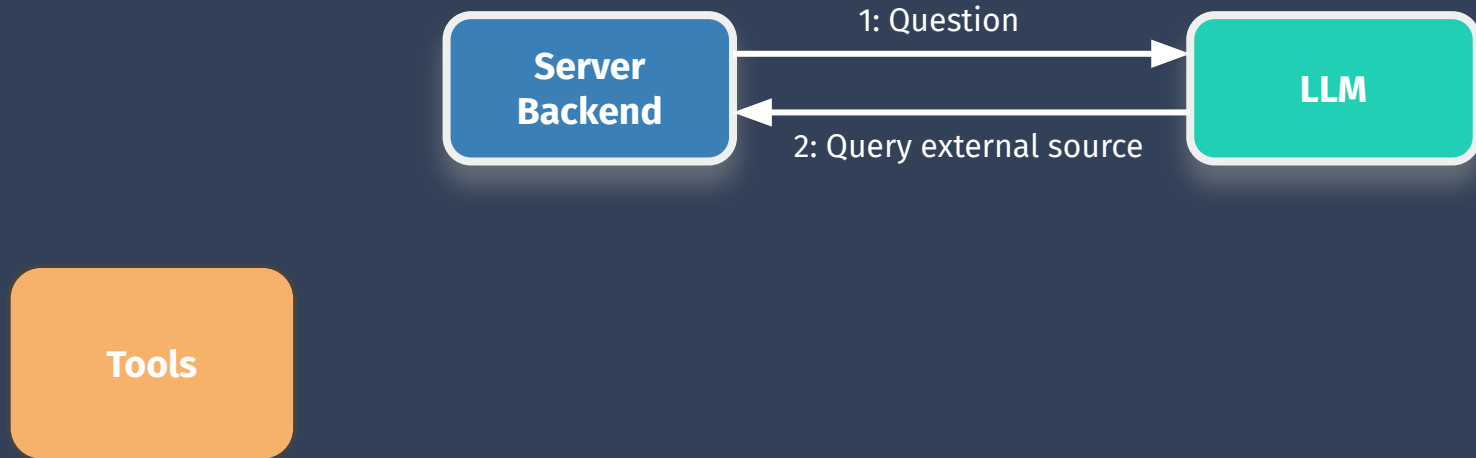
Deploy Your GenAI Project

Making it LIVE.....

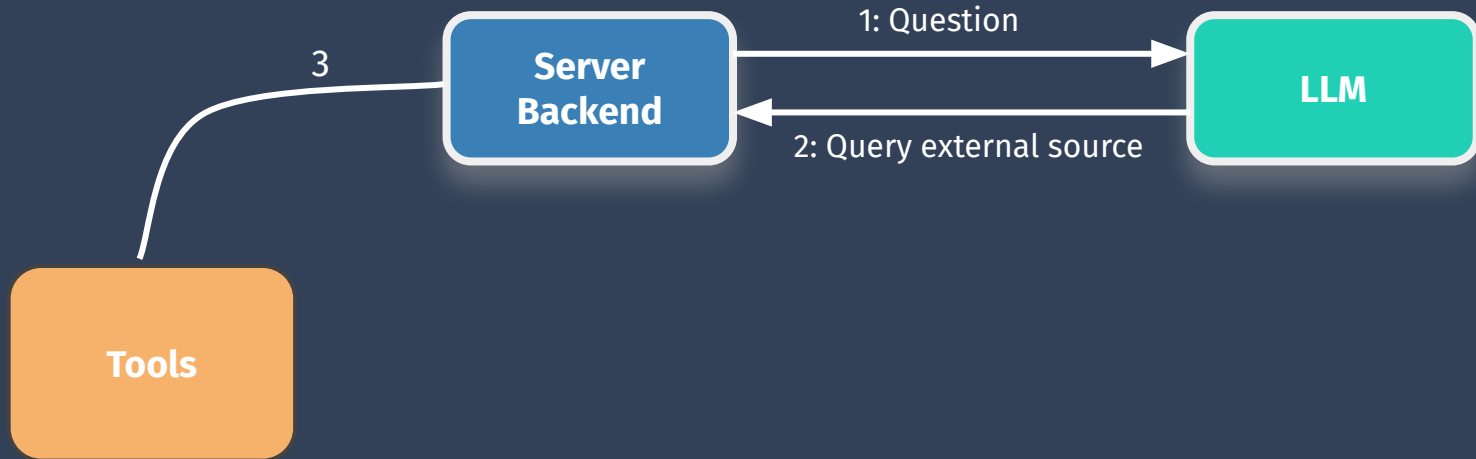
Agents with Tools



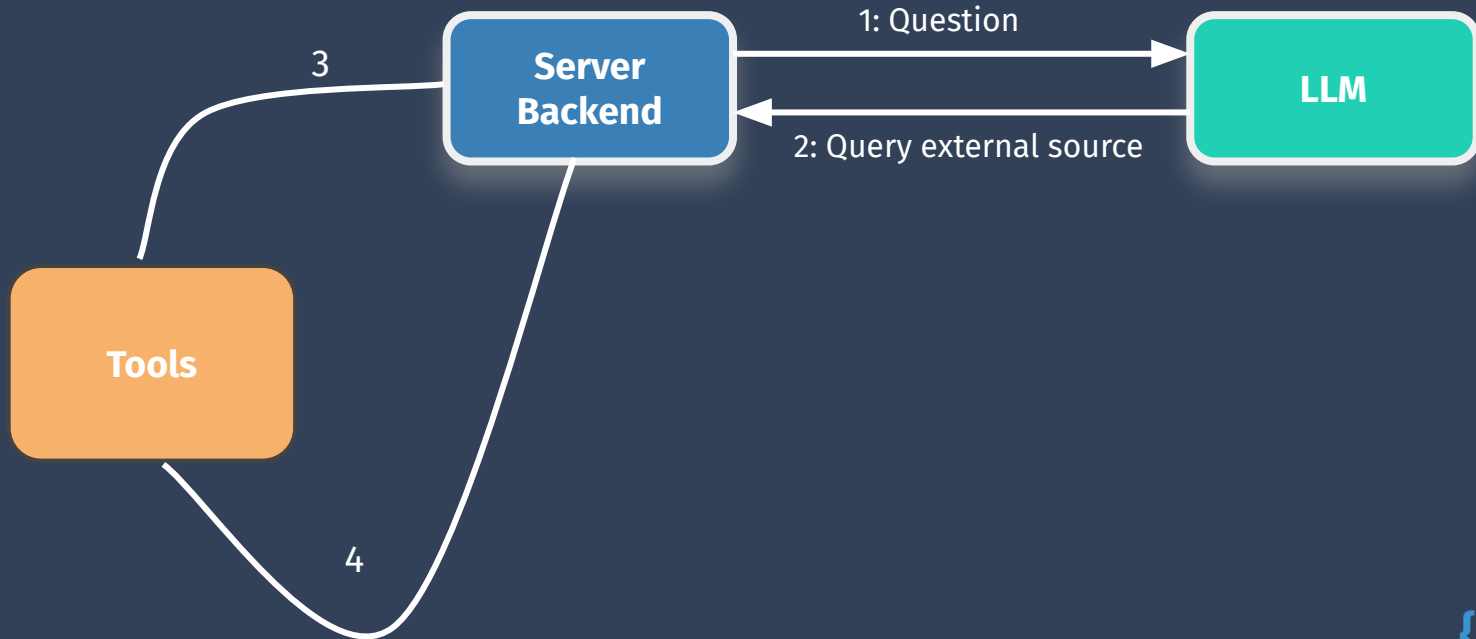
Agents with Tools



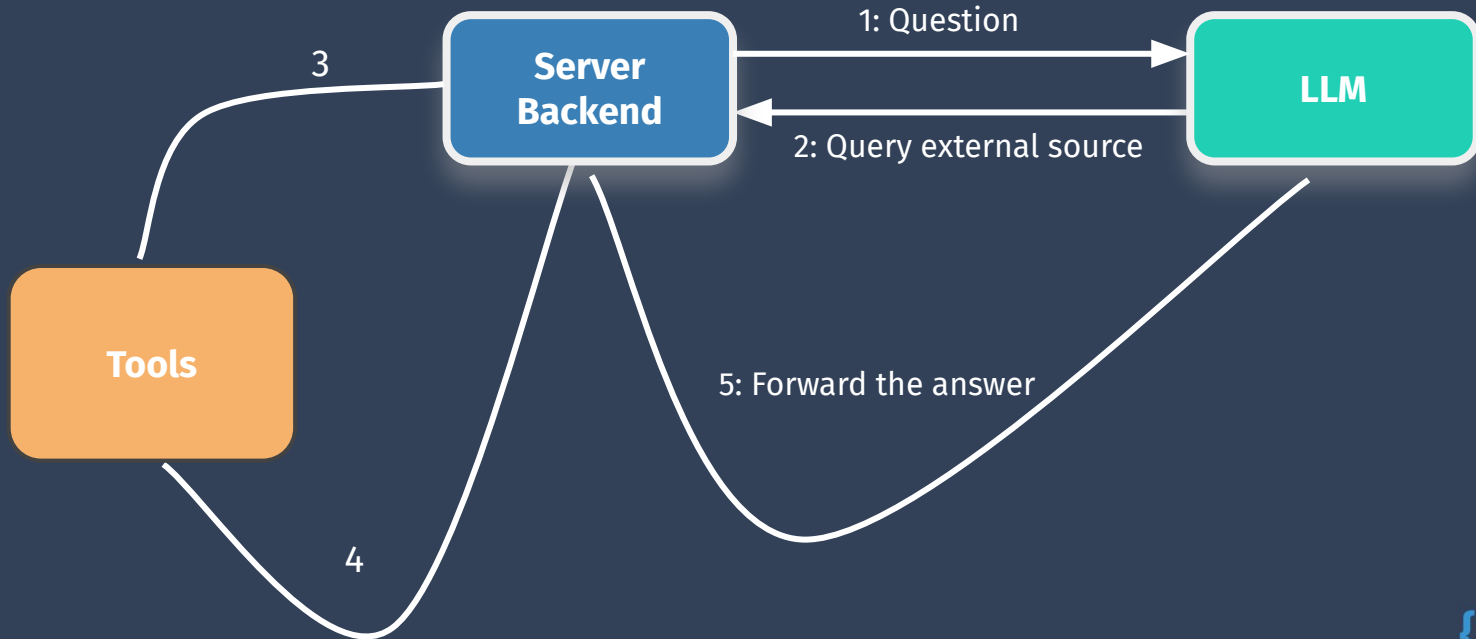
Agents with Tools



Agents with Tools

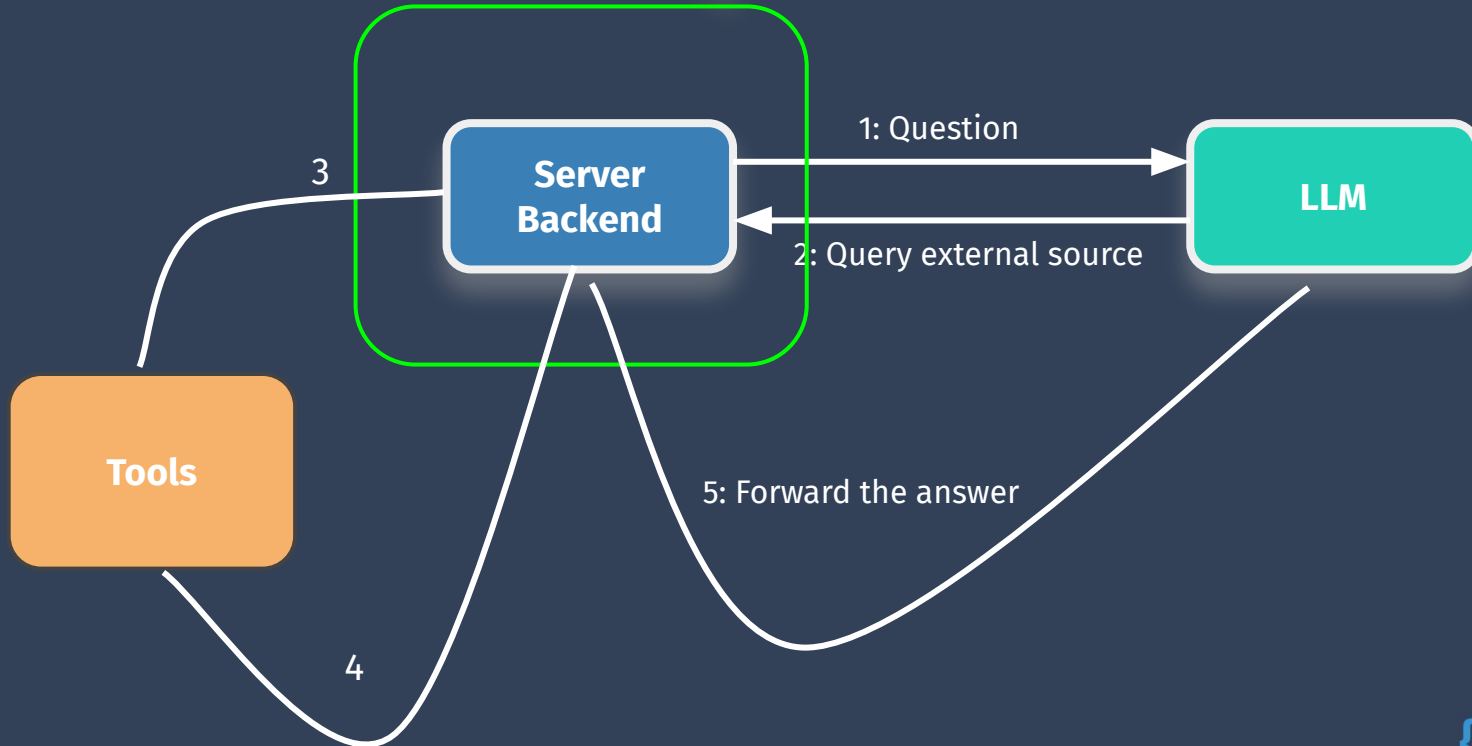


Agents with Tools



Agents with Tools

This requires building and deploying



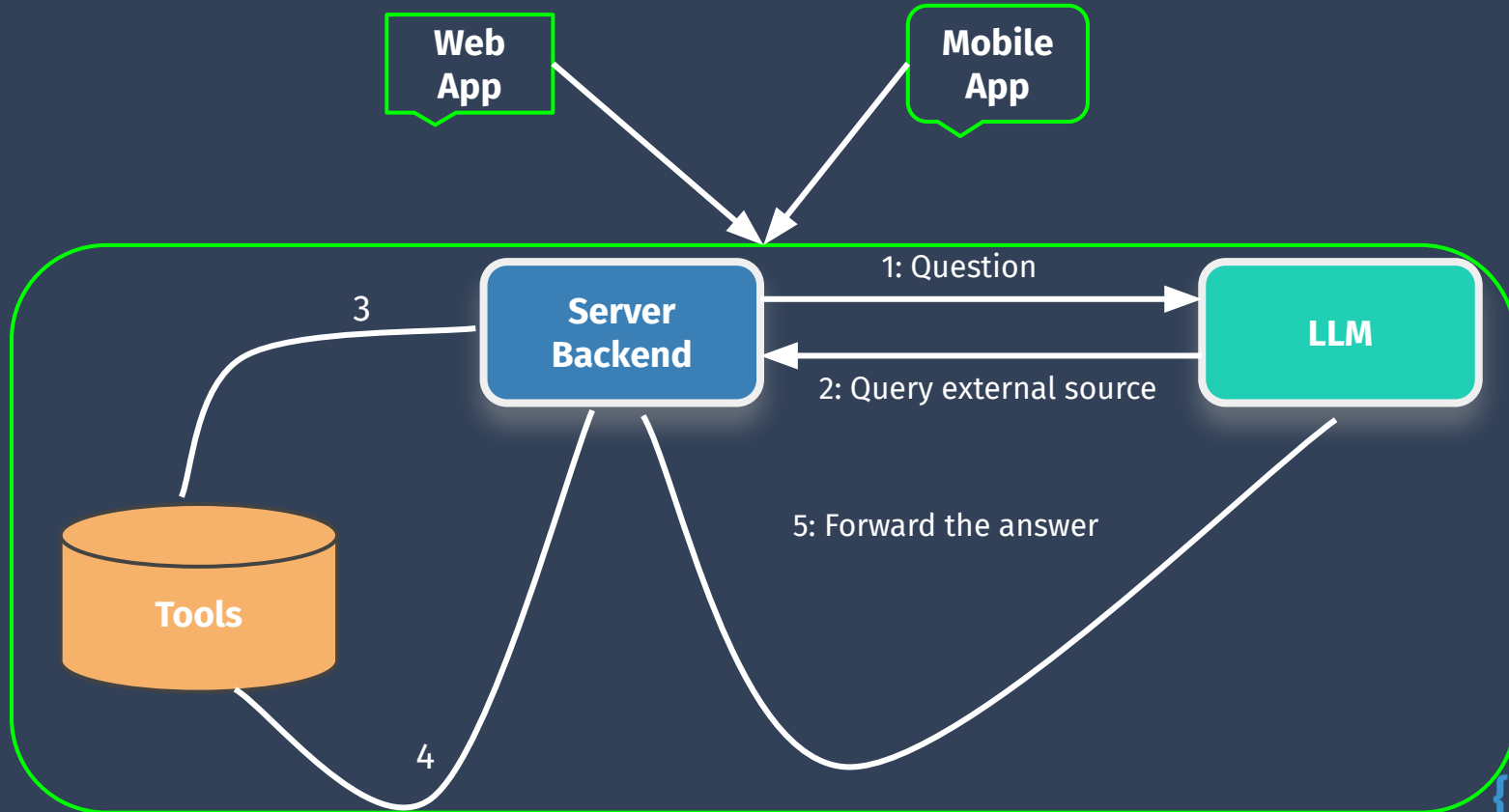
What is Deployment?

Deployment is the process of making your chatbot accessible for real-world use by **hosting it on a server or cloud platform**. It involves:

- Setting up the backend
- Integrating APIs
- Ensuring that users can interact with the chatbot through:
 - Web app
 - Mobile app
 - Or other interfaces.



Agents with Tools



Why do we need it?



Real-World Usage: Allows users to interact with your chatbot beyond your local system.



Performance Testing: Helps you monitor real-time usage, performance, and improvements.



Scalability: Deploying ensures that your chatbot can handle multiple users efficiently.



Portfolio & Experience: Showcasing a deployed project boosts your credibility as a developer.

Deployment Options



Hugging Face



Streamlit

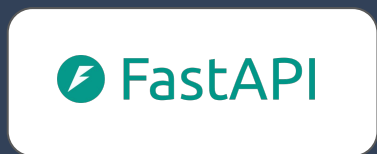


Google Cloud



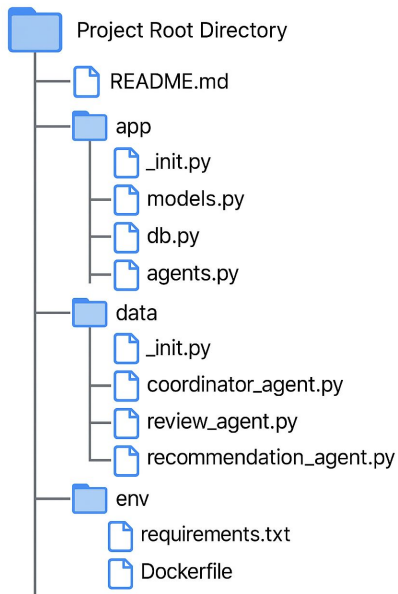
netlify

Different Ways to Deploy Your Project



Covered Today !!!

Upload the files



Project Structure

FastAPI

- FastAPI is used here to handle real-time interactions with our AI models via RESTful APIs.
- This enables external systems (such as a web app or a chatbot UI) to send and receive data to/from the AI model seamlessly.



Writing the FastAPI Code

FastAPI is a modern, fast (high-performance) web framework for building APIs with Python 3.7+ based on standard Python type hints.

It is built on top of Starlette for the web parts and Pydantic for the data parts. FastAPI is designed to be easy to use, flexible, and robust for building APIs.

Why is FastAPI Needed Here?

- **Performance** - FastAPI allows quick API creation with minimal code and handles asynchronous tasks efficiently.
- **Speed** - One of the fastest Python web frameworks, optimized for scaling applications.
- **Automatic Validation** - Automatically validates requests using Pydantic, ensuring proper data formats.
- **Automatic Documentation** - Auto-generates interactive API docs (Swagger UI & ReDoc) for easier testing and consumption.

Writing the FastAPI Code

```
from fastapi import FastAPI
from pydantic import BaseModel
from crewai import Crew, Task

app = FastAPI()

class QueryInput(BaseModel):
    query: str

@app.post("/query")
async def handle_query(input: QueryInput):
    task = Task(description=f"Process: {input.query}",
agent=router_agent, agents=[router_agent, agent_1, agent_2], tasks=[task])
    result = crew.kickoff()
    return {"response": [t.raw for t in result.tasks_output]}
```

Multi-Agent Systems

A single agent might struggle if it needs to specialize in multiple domains or manage many tools.

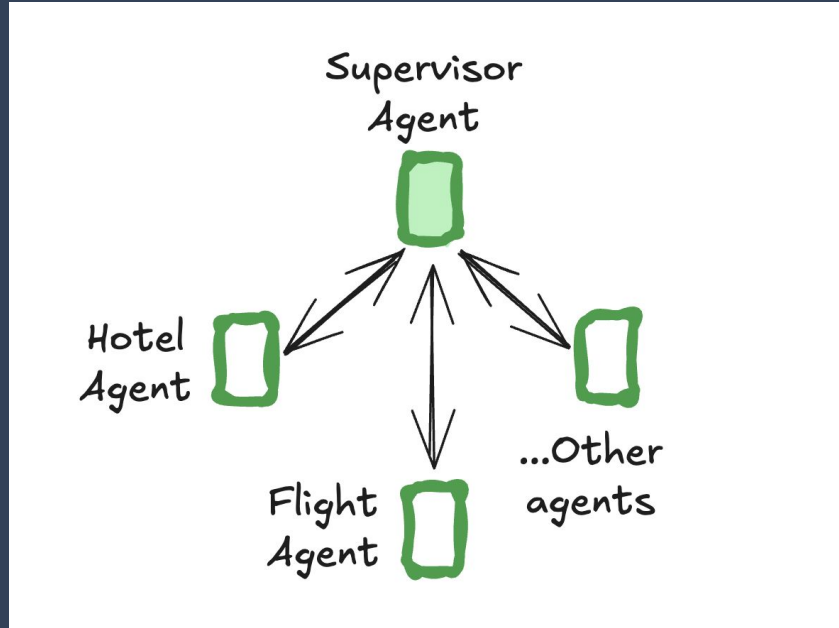
To tackle this, you can break your agent into smaller, independent agents and composing them into a multi-agent system.

Two of the most popular multi-agent architectures are:

1. supervisor
2. swarm

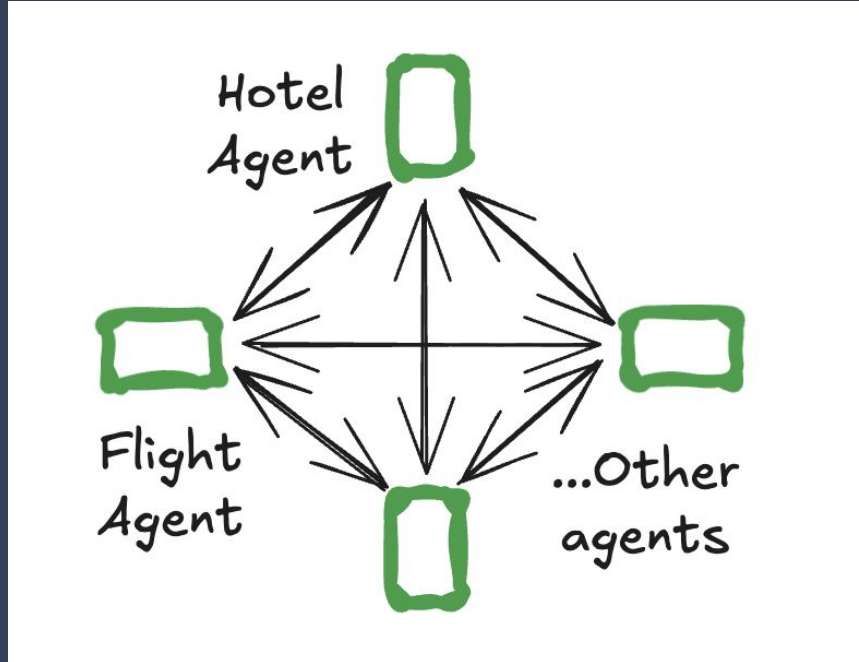
Multi-Agent Systems - Supervisor

Individual agents are coordinated by a central supervisor agent. The supervisor controls all communication flow and task delegation, making decisions about which agent to invoke based on the current context and task requirements.



Multi-Agent Systems - Swarm

Agents dynamically hand off control to one another based on their specializations. The system remembers which agent was last active, ensuring that on subsequent interactions, the conversation resumes with that agent.



Building The DockerFile

Step 1: Start from a Python base image

- Use python:3.11-slim for lightweight and compatibility.

Step 2: Set working directory & copy code

- Copy backend code and your static/ folder containing HTML/CSS.

Step 3: Install Python dependencies

- Include FastAPI, uvicorn, crewai, etc.

Step 4: Expose ports and launch the app

- Use CMD to run the FastAPI app with uvicorn.

Building The DockerFile

```
# Base image
FROM python:3.11.11

# Set working directory
WORKDIR /app

# ✅ Create writable directory for CrewAI storage inside `/src/data`
RUN mkdir -p /src/data && chmod -R 777 /src/data

# Copy all project files
COPY . .

# Install dependencies
RUN pip install --no-cache-dir -r requirements.txt

# Expose FastAPI Port
EXPOSE 7860

# Run FastAPI
CMD ["uvicorn", "src.app:app", "--host", "0.0.0.0", "--port", "7860"]
```

Done **Uploading!!!!**

What next?



Time for Deployment!!!

Live Zoom Poll

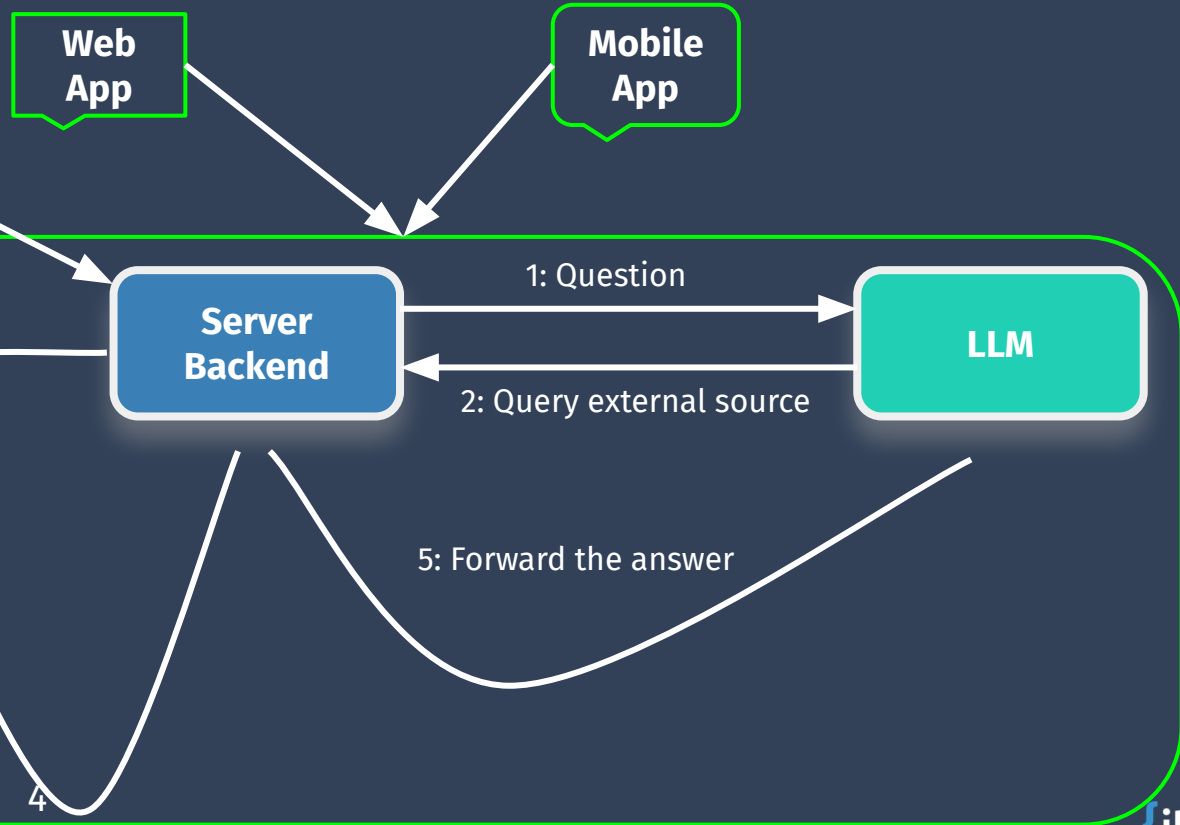
We are launching a poll on Zoom. Please take 2 minutes to fill out the form.

Let's look into the
steps on how to deploy your project



Agents

Deployed on
AWS



Step 1 : Set Security Config

Allow TCP calls to the endpoint

Inbound rules control the incoming traffic that's allowed to reach the instance.

Inbound rules	Type	Protocol	Port range	Source	Description - optional	
Security group rule ID sgr-07409d3c52fc6221e	Custom TCP	TCP	7860	Custom 0.0.0.0/0		Delete
sgr-098f957797dff042	HTTPS	TCP	443	Custom 0.0.0.0/0		Delete
sgr-05b69ad4f4999cc3a	SSH	TCP	22	Custom 0.0.0.0/0		Delete
sgr-0d038182e600e34ed	HTTP	TCP	80	Custom 0.0.0.0/0		Delete

[Add rule](#)

Rules with source of 0.0.0.0/0 or ::/0 allow all IP addresses to access your instance. We recommend setting security group rules to allow access from known IP addresses only.

[Cancel](#) [Preview changes](#) [Save rules](#)

Step 2 : Setup EC2 Instance

1. Create EC2 Instance
2. Create new key pair
 - a. You cannot re-download it and you cannot use a different key pair
3. Create EC2 instance “Deep Learning OSS Nvidia Driver AMI GPU PyTorch 2.6 (Ubuntu 22.04)”
4. Associate the key pair from step 1

Step 3 : Connect to EC2 Instance

- Go to “Connect”
- Then choose “EC2 Instance Connect”

The screenshot shows the AWS Management Console interface for an EC2 instance. At the top, a green banner indicates 'Successfully initiated starting of i-Od2bb7553797d973c'. The left sidebar contains navigation links for EC2, Instances, and various instance-related services. The main content area displays the 'Instance summary for i-Od2bb7553797d973c (dep)'. The instance is in a 'Running' state. A red box highlights the 'Connect' button in the top right corner of the instance details panel. Below the 'Connect' button are 'Instance state' and 'Actions' buttons. The instance details are organized into columns: Instance ID, Public IPv4 address, Private IPv4 addresses, Public IPv4 DNS, Elastic IP addresses, AWS Compute Optimizer finding, Auto Scaling Group name, Managed, Instance type, VPC ID, Subnet ID, Instance ARN, and Operator.

Instance ID	Public IPv4 address	Private IPv4 addresses	Public IPv4 DNS	Elastic IP addresses	AWS Compute Optimizer finding	Auto Scaling Group name	Managed	Instance type	VPC ID	Subnet ID	Instance ARN	Operator
i-Od2bb7553797d973c	54.227.59.87 open address	172.31.26.171	ec2-54-227-59-87.compute-1.amazonaws.com open address	-	Opt-in to AWS Compute Optimizer for recommendations. Learn more	-	false	t2.micro	vpc-00dc6e082eb4f6d6f	subnet-0fcb65c45bee0f94c	arn:aws:ec2:us-east-1:720624239208:instance/i-Od2bb7553797d973c	-

Step 4 : Alternatively SSH to the EC2 instance (Remote Machine)

From your terminal run

- `chmod 400 PATH_TO_PEM_FILE`
- `ssh YOUR_INSTANCE_ADDRESS`

You can get access to your instance address from the “Connect” tab

EC2 > Instances > i-0d2bb7553797d973c > Connect to instance

Connect to instance Info

Connect to your instance i-0d2bb7553797d973c (dep) using any of these options

EC2 Instance Connect | Session Manager | **SSH client** | EC2 serial console

Instance ID
i-0d2bb7553797d973c (dep)

1. Open an SSH client.
2. Locate your private key file. The key used to launch this instance is genai_dep.pem
3. Run this command, if necessary, to ensure your key is not publicly viewable.
`chmod 400 "genai_dep.pem"`
4. Connect to your instance using its Public DNS:
`ec2-54-227-59-87.compute-1.amazonaws.com`

Example:
`ssh -i "genai_dep.pem" ubuntu@ec2-54-227-59-87.compute-1.amazonaws.com`

Note: In most cases, the guessed username is correct. However, read your AMI usage instructions to check.

List of Commands to run from the remote Machine

Once you are the terminal of the EC2 instance (either through ssh or instance connect”) run the following

- Download the code, run:
`git clone https://huggingface.co/spaces/Abhishek306/healthcheck-app`
- Run: `cd <REPO_NAME>`
 - # Example: `cd healthsense-main-new`
- Run: `python3.11 -m venv deploy_genai_venv`
- Run: `source deploy_genai_venv/bin/activate`
- Run: `pip install -r requirements.txt`
- Run:
 - `mkdir ~/src`
 - `mkdir ~/src/data`
- Change line 46 in `app.py` `~/src/data`
- Run: `python -m src.app`

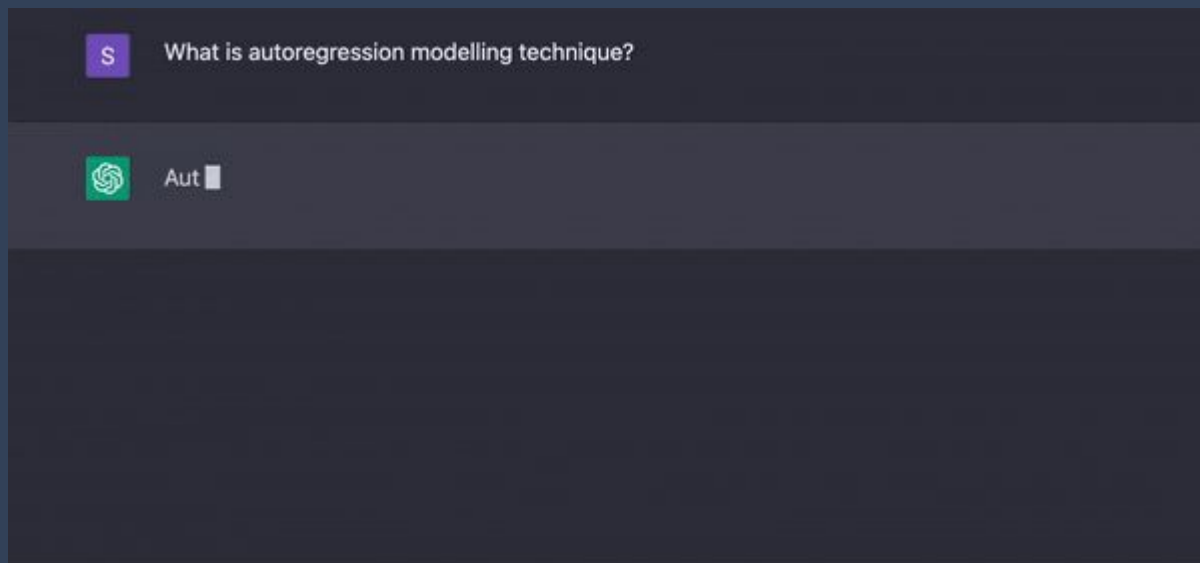
To View the Application

Open your browser then type: <public ip address>:7860/#

Time for Deployment!!!

Optimizing LLM

Token Streaming



How do you stream data to the client side?

- Polling
 - The client side keeps asking the backend for any new tokens available

How do you stream data to the client side?

- Polling
 - The client side keeps asking the backend for any new tokens available
- Server-side events
 - Pushing data to the client as soon as it is available.
 - More efficient than polling because it eliminates unnecessary network traffic.
 - Uni-directional, where the server is dispatching data to the client-side outside of the initial request

How do you stream data to the client side?

- Polling
 - The client side keeps asking the backend for any new tokens available
- Server-side events
 - Pushing data to the client as soon as it is available.
 - More efficient than polling because it eliminates unnecessary network traffic.
 - Uni-directional, where the server is dispatching data to the client-side outside of the initial request
- Websockets
 - Similar to server-side events but bi-directional
 - Common choice for something communication-based, like a webchat or audio call.

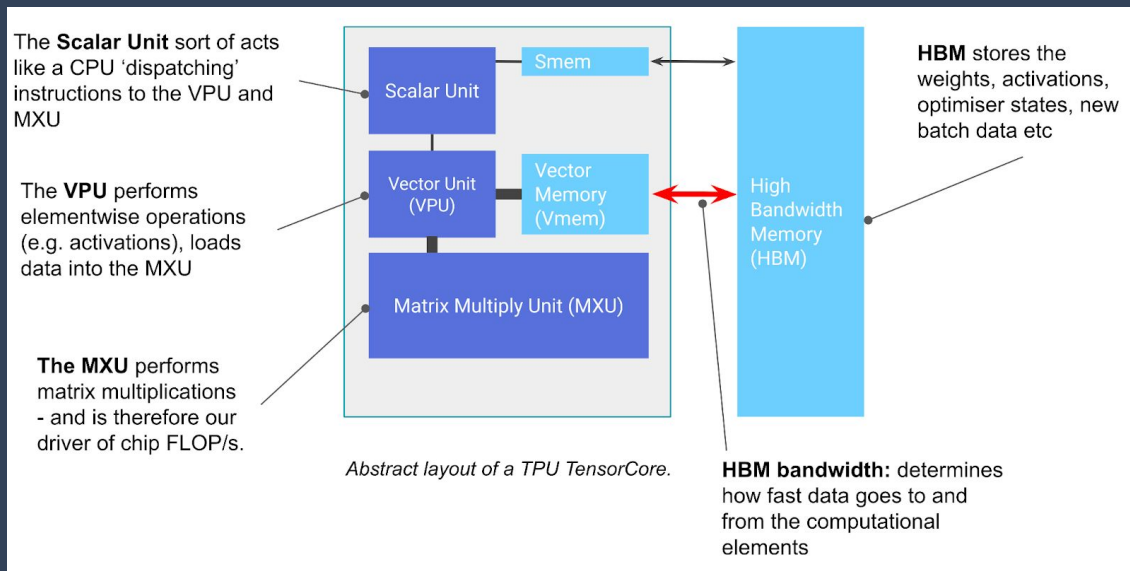
How do you stream data to the client side?

- Polling
 - The client side keeps asking the backend for any new tokens available
- Server-side events
 - Pushing data to the client as soon as it is available.
 - More efficient than polling because it eliminates unnecessary network traffic.
 - Uni-directional, where the server is dispatching data to the client-side outside of the initial request
- Websockets
 - Similar to server-side events but **bi-directional**
 - Common choice for something communication-based, like a webchat or audio call.

Batching

LLMs are memory-bound:

- Many cycles are wasted moving data from memory to processing unit



Batching

Instead of loading new model parameters each time you have an input sequence
Load the model parameters only once and then use them to process many input sequences.

T_1	T_2	T_3	T_4	T_5	T_6	T_7	T_8
S_1	S_1	S_1	S_1				
S_2	S_2	S_2	S_2				
S_3	S_3	S_3	S_3				
S_4	S_4	S_4	S_4	S_4			

T_1	T_2	T_3	T_4	T_5	T_6	T_7	T_8
S_1	S_1	S_1	S_1	S_1	END		
S_2	S_2	S_2	S_2	S_2	S_2	S_2	END
S_3	S_3	S_3	S_3	END			
S_4	S_4	S_4	S_4	S_4	S_4	END	

 Input prompt
 Output tokens
 End-of-sequence token

Advanced Batching

Outside the scope:

<https://insujang.github.io/2024-01-07/llm-inference-continuous-batching-and-pagedattention/>

T_1	T_2	T_3	T_4	T_5	T_6	T_7	T_8
S_1	S_1	S_1	S_1				
S_2	S_2	S_2					
S_3	S_3	S_3	S_3				
S_4	S_4	S_4	S_4	S_4			

T_1	T_2	T_3	T_4	T_5	T_6	T_7	T_8
S_1	S_1	S_1	S_1	S_1	END	S_6	S_6
S_2	S_2	S_2	S_2	S_2	S_2	S_2	END
S_3	S_3	S_3	S_3	END	S_5	S_5	S_5
S_4	S_4	S_4	S_4	S_4	S_4	END	S_7

 Input prompt
 Output tokens
 End-of-sequence token

Model-level Optimization

How much memory does it need to run a model?

The rough rule of thumb is:

- Each parameter ≈ 2 bytes (FP16)
- Total VRAM needed = model size $\times 2$ bytes $\times$ “activation + overhead (20–30%)”

7B model

- Parameters: 7 billion
- FP16: $7\text{B} \times 2 = \mathbf{14\text{ GB}}$ for weights alone
- Add ~30% overhead for activations, KV cache, etc.
- **Total VRAM needed $\approx 20\text{ GB}$**
-  Can run on **RTX 3090 (24 GB)** or **A10 (24 GB)**, or any larger GPU

70B model

- Parameters: 70 billion
- FP16: $70\text{B} \times 2 = \mathbf{140\text{ GB}}$
- With overhead: **200 GB VRAM**
-  Needs at least **two A100 80GB**, or one **B100 (192 GB)** (barely fits)

Optimizing Inference Speed & Model Costs

1. Smaller == Faster

- Quantization -> Smaller Models
- Model Pruning
- Distillation

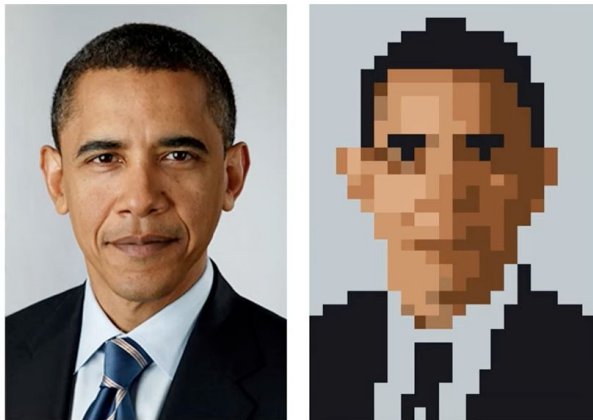
2. Inference Optimizations Techniques

- Caching Results
- Batching
- Speculative Decoding
- Software Enhancements

3. Deployment

- Model picking
- Hardware/Model match
- Pay for what you use
- Inference Stack Options

A. Quantization



FP32: [33.623563422, 12.646104098, -51.583920991, ...]

FP16: [33.6234, 12.6461, -51.5839, ...]

INT8: [82, 44, -23, ...]

Intuition: Can we save space by lowering precision of parameters?

A. Quantization

Quantization is like switching from full words to emojis in texting

Imagine you're texting a friend:

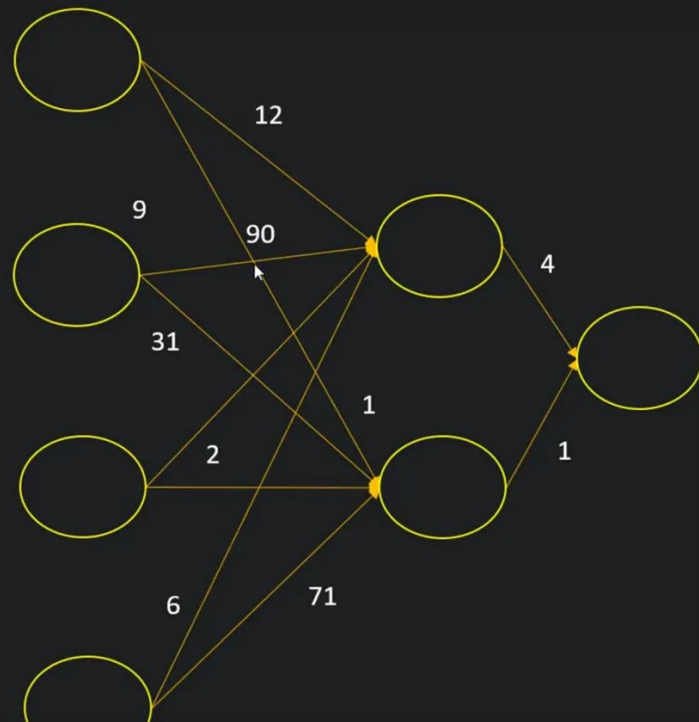
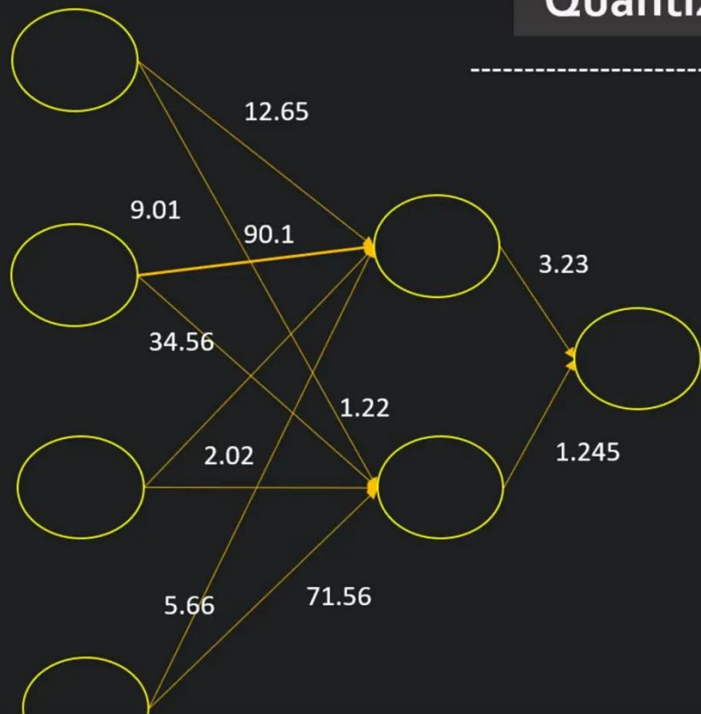
- **Full-precision LLM (FP16/FP32)** is like writing full sentences:
"I'm feeling really happy and excited today!"
- **Quantized LLM (e.g. INT8)** is like sending:
"😊🎉"

Both convey the **same idea**, but:

- Emojis take **less space** (storage/memory)
- They're **faster to send and receive**
- But you might lose some **nuance** or **precision**

Model Quantization

Quantization



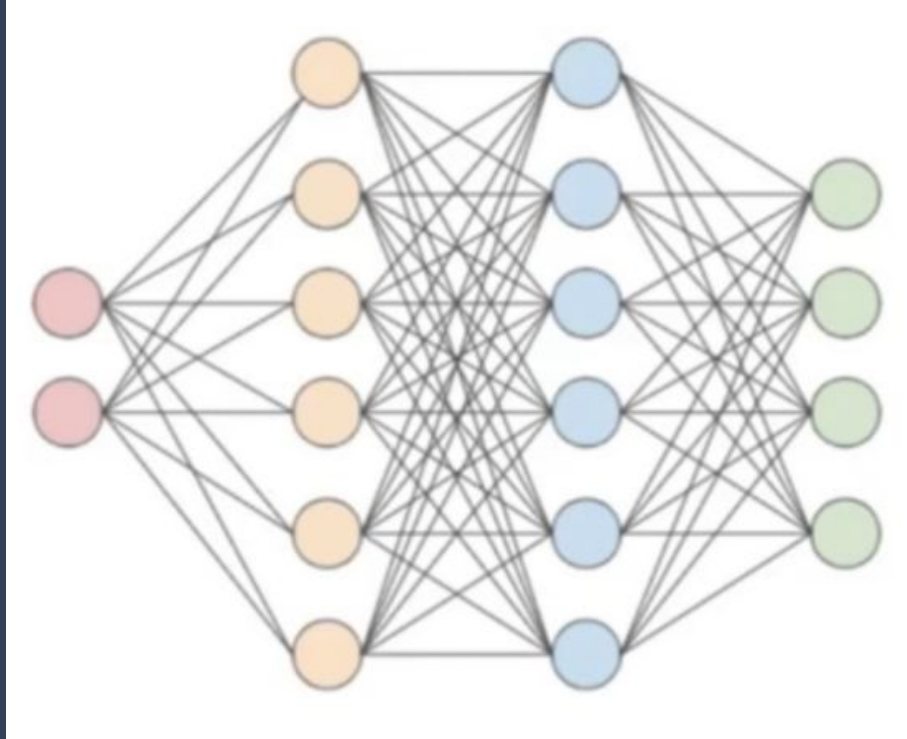
A. Quantization

Concept: Reducing the numerical precision of a model's weights. Most models trained using float32 (precise but computationally intensive). Less precise formats like float16 or int8, we can achieve substantial gains.

Why it Works:

- **Smaller Memory Footprint:** int8 model 4x smaller than float32 counterpart. Less RAM/VRAM needed, model loads faster.
- **Faster Data Transfer:** Moving data types from memory to the compute (CPU/GPU) quicker.
- **Faster Computation:** Modern CPUs/GPUs perform integer operations much faster than floating-point.

B. Model Pruning



B. Model Pruning

Model Pruning = Trimming a Bonsai Tree

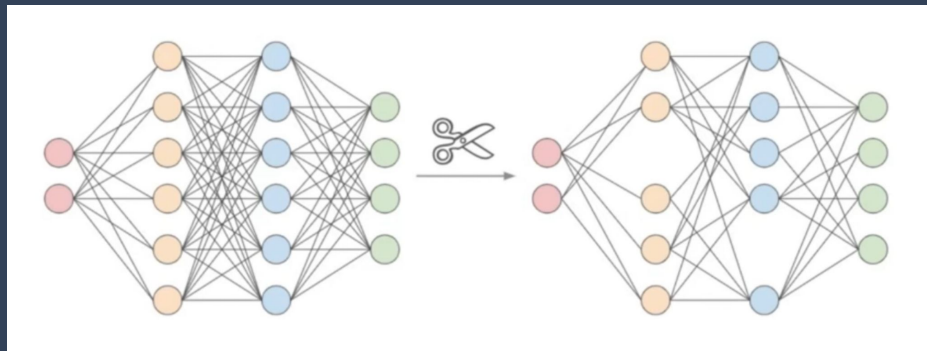
Imagine you're growing a bonsai tree:

- At first, it has **too many branches**.
- You **carefully snip away** the ones that don't contribute to shape or health.
- The result is **smaller, cleaner**, and often **just as beautiful**.

That's pruning in LLMs:

- You remove weights or neurons that don't contribute much to output.
- The model becomes smaller and faster.
- If done right, accuracy stays the same (or drops very slightly).

Intuition: Can I set some weights to ZERO to save storage and compute?



B. Model Pruning

- **Methods:**

- **Unstructured Pruning:** Removes individual weights, creating "sparse" models. Require special hardware or software libraries.
- **Structured Pruning:** Removes entire groups of weights, such as neurons or attention heads. Works on standard hardware, making it more practical.

- **Verdict:**

- Use unstructured pruning for max compression and have the right (special) infrastructure.
- Use structured pruning for real-world latency or throughput gains on typical hardware (GPUs with sparse tensor core support A100, H100)

Model Pruning

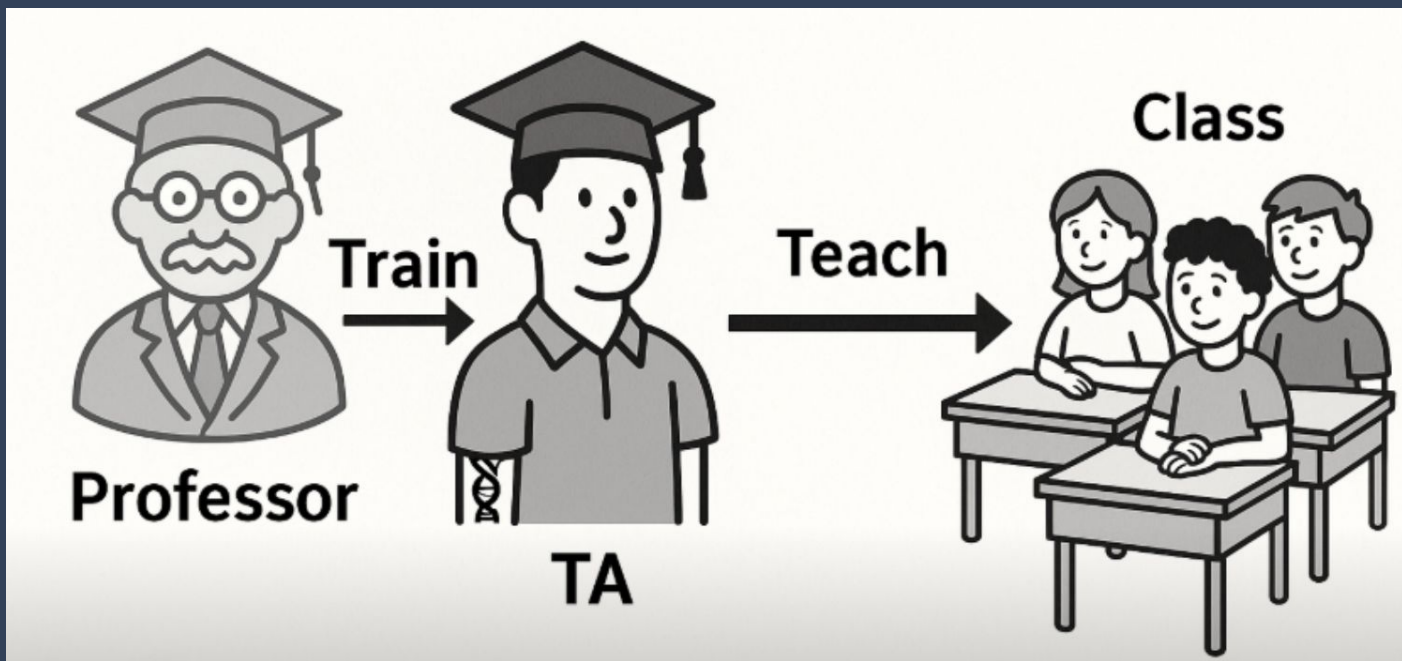
- At training time
 -   More efficient as the model is trained with sparsity at training time
 -   Can be more complex
- Post training time
 -   Simpler to implement and adjust per use-case
 -   May require fine-tuning

C. Distillation

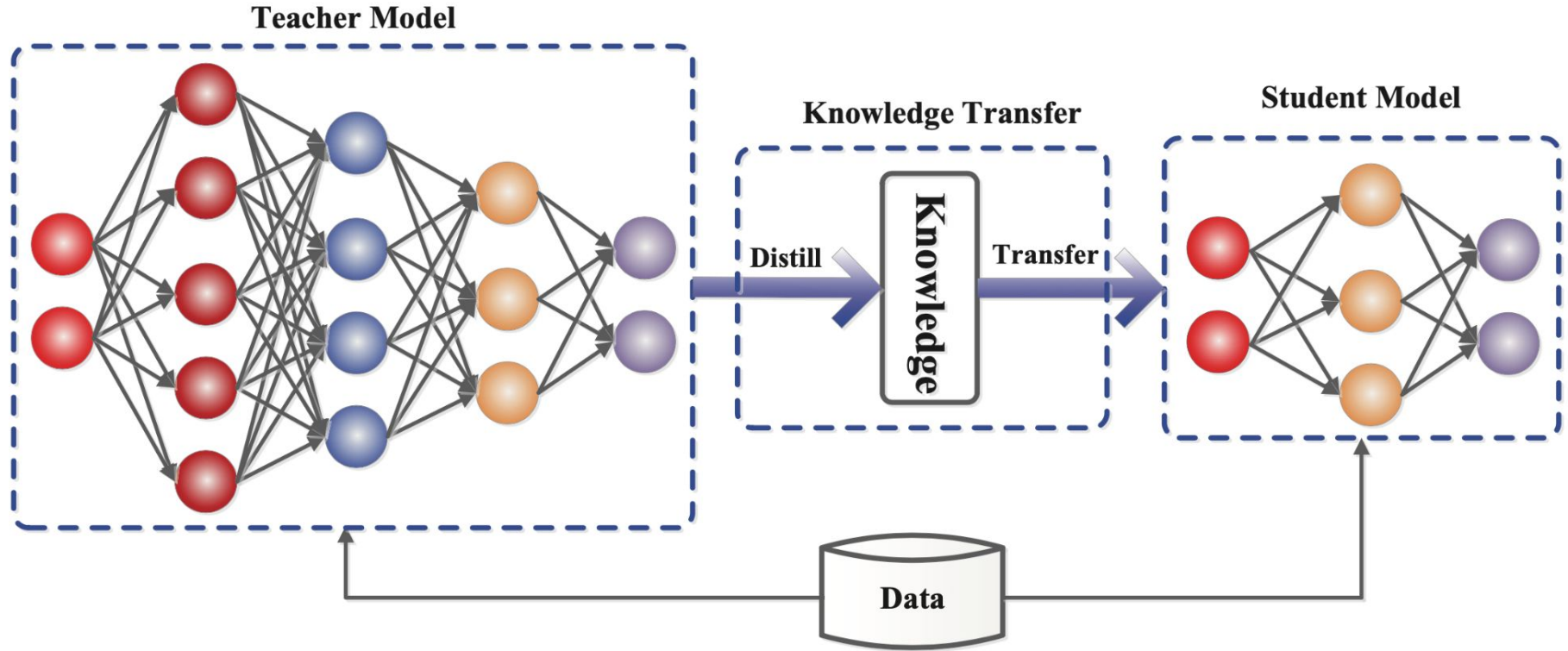


C. Distillation

Intuition: I don't need a professor to teach high school math



Knowledge Distillation

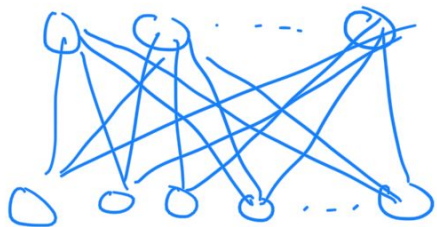


Distillation

Honda SUV chair
0.6 0.35 0.05



...

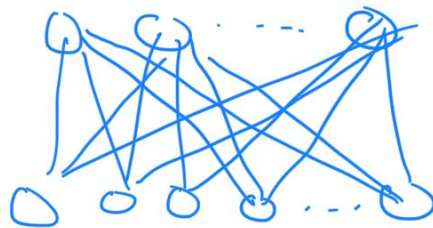


i/p img 1

Honda SUV chair
0.39 0.59 0.02



...



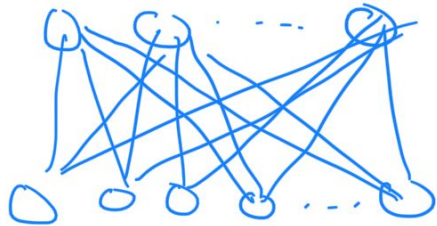
i/p img 2

Distillation

student learner player
0.6 0.35 0.05



...

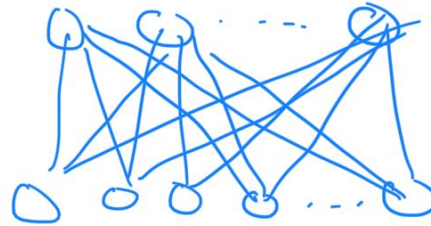


i/p txt 1

student learner player
0.39 0.59 0.02

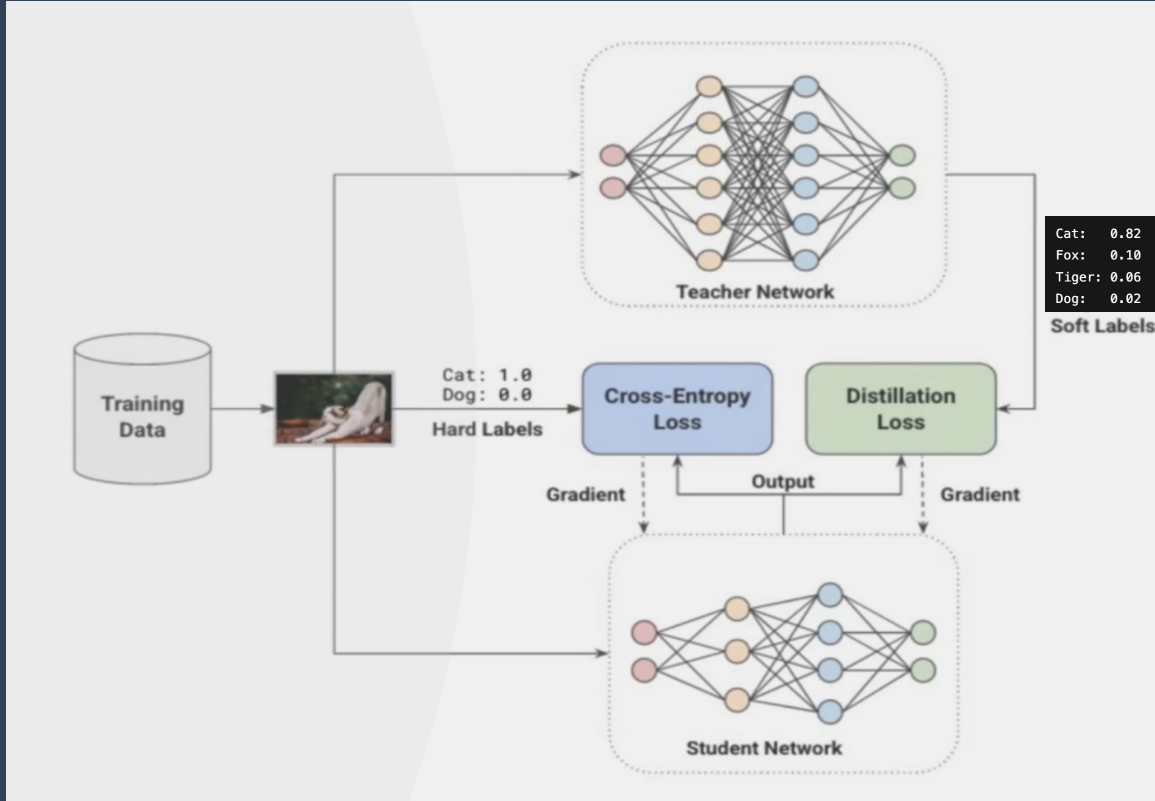


...



i/p txt 2

C. Distillation



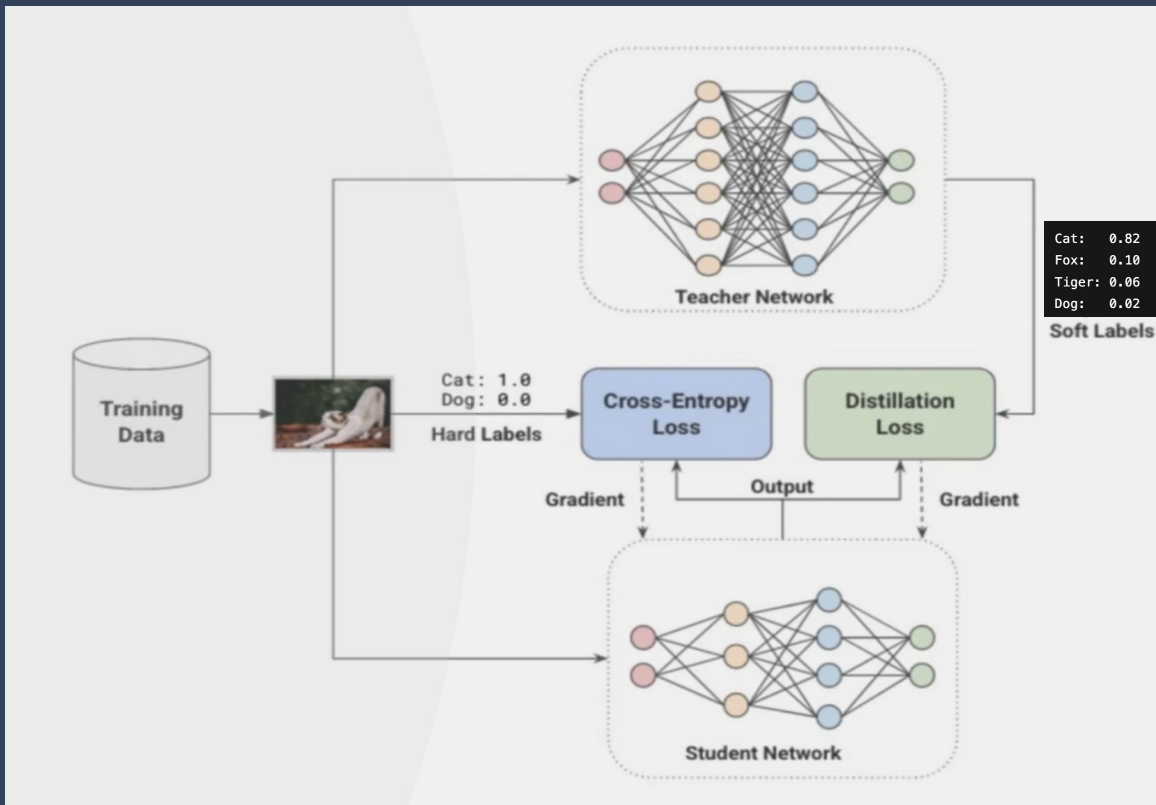
Training data is passed to both:

1. The **teacher network** (pretrained, large model)
2. The **student network** (smaller model we want to train)

Backpropagate:

1. Distillation Loss, teaching student to mimic teacher's behavior

C. Distillation



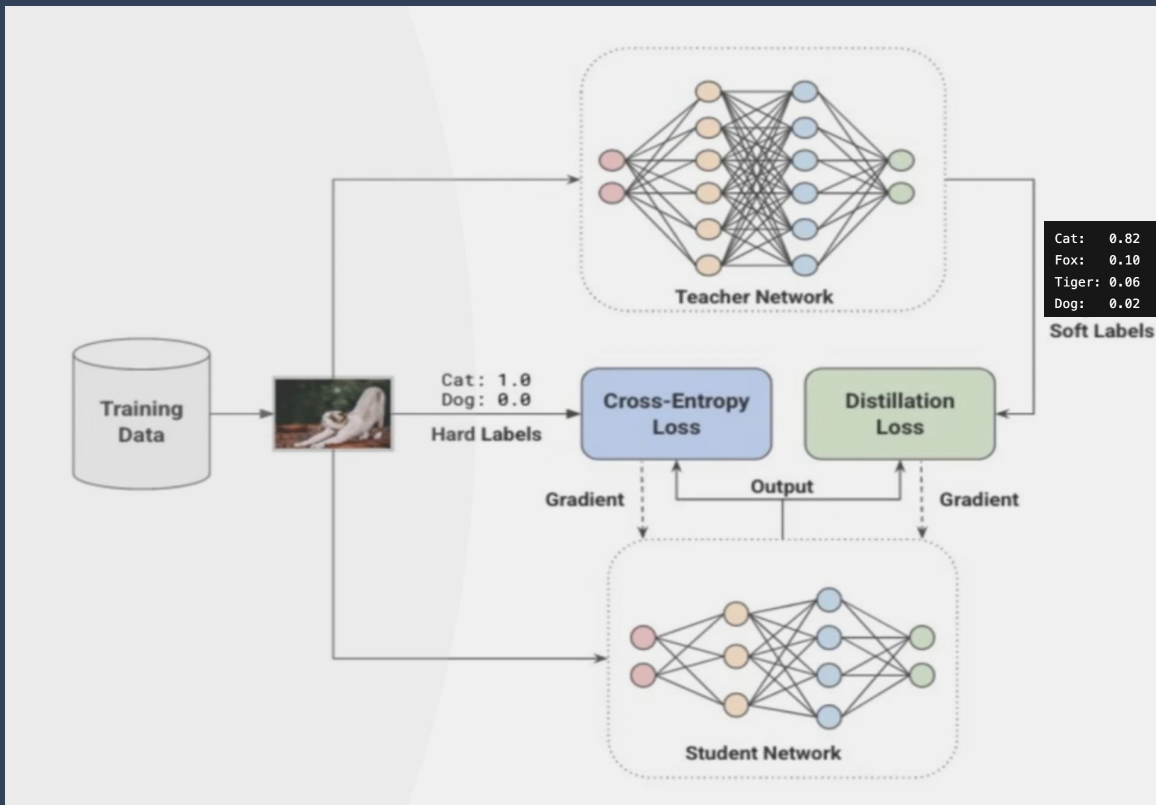
Training data is passed to both:

1. The **teacher network** (pretrained, large model)
2. The **student network** (smaller model we want to train)

Use 2 gradients to backpropagate:

1. Distillation Loss, teaching student to mimic teacher's behavior
2. Cross-Entropy loss, forcing students to match true labels

C. Distillation



Training data is passed to both:

1. The **teacher network** (pretrained, large model)
2. The **student network** (smaller model we want to train)

Use 2 gradients to backpropagate:

1. Cross-Entropy loss, forcing students to match true labels
2. Distillation Loss, teaching student to mimic teacher's behavior

Why not train student model directly?

- Output of teacher contain more information, and can teach faster than pure labels

Distillation in LLM

- DistilBERT
 - The student model imitate the teacher model
 - Has about half the total number of parameters of BERT base and retains 95% of BERT's performances on the language understanding benchmark

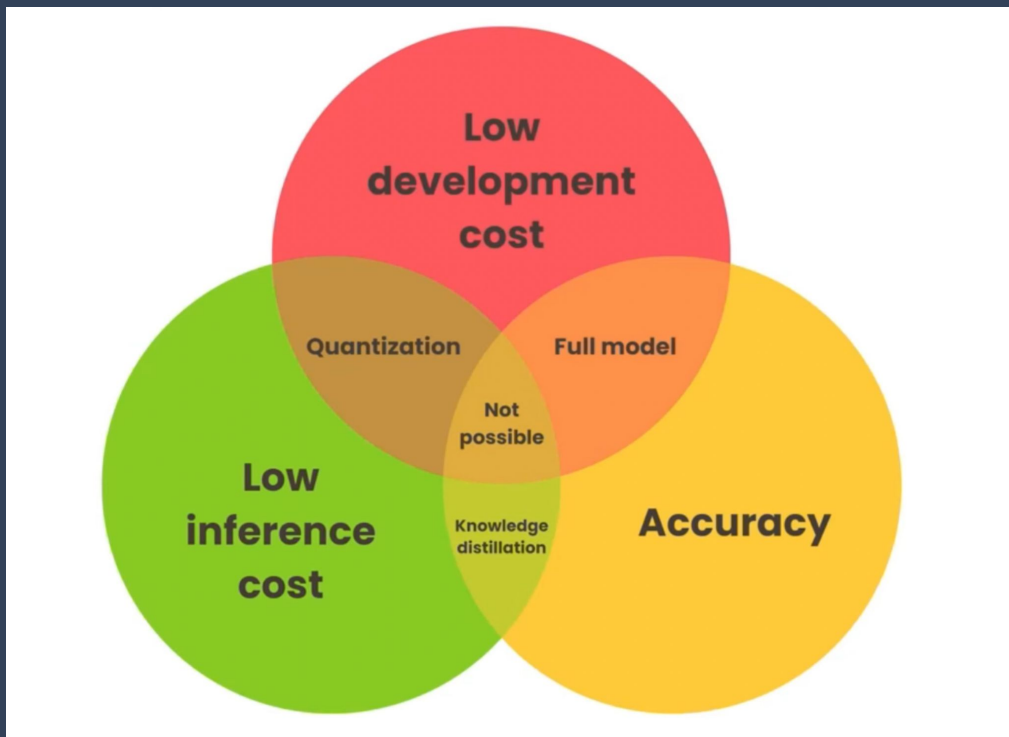
Distillation: Practical Considerations

- The student model will be good at the distillation task **only**
- You still need unlabeled data
 - In some enterprise setups you may not be allowed to use customer data
- Model refresh

Summary of Optimizations

Optimization Type	Summary
Quantization	Use less precise data formats to reduce model size and latency up to 4x.
	Best combined with reduced precision execution engine.
	Potential loss in accuracy.
Pruning	Sets some weights to zero to save space and compute.
	Requires sparse execution engine.
	Potential loss in accuracy.
Distillation	Able to modify the model architecture.
	Expensive to train.

Tradeoffs in Model Deployment



You can't have all three: Accuracy, low dev cost, and low inference cost — choose two.

Full Model

- Highest accuracy
- Minimal dev time (just fine-tune)
- But **expensive to deploy**

Quantization

- Lowers inference cost by compressing weights (e.g., FP32 → INT8)
- Preserves dev simplicity
- But can **hurt accuracy**

Knowledge Distillation

- Train a small model to mimic a large one
- Balances **accuracy** and **inference cost**
- But **requires extra training effort**

Demo Time

<https://colab.research.google.com/drive/15MyXtLieUTt2dLbtQsvL84BN8rgGkmoX#scrollTo=id9p0oxlblS4>



Quiz

Distillation Improves:

- A) Training latency
- B) Inference latency
- C) A & B

Quiz

Distillation Improves:

- A) Training latency
- B) Inference latency
- C) A & B



Quiz

Model Pruning and Quantization Improve

- A) Training latency
- B) Inference latency
- C) A & B

Quiz

Model Pruning and Quantization Improve

- A) Training latency
- B) Inference latency
- C) A & B

**Thanks
Questions?**

References

- <https://www.vellum.ai/llm-parameters/llm-streaming>
- <https://huggingface.co/docs/text-generation-inference/en/conceptual/streaming>
- <https://jax-ml.github.io/scaling-book/tpus/>
- <https://medium.com/@yohoso/llm-inference-optimisation-continuous-batching-2d66844c19e9>
- https://huggingface.co/docs/transformers/en/perf_train_gpu_many